Supplementary Material for "Super-Precise Latency: How CPU Threading Affects High-Precision Latency, and an Industry-Wide Audit"

Gustavo Pedro Ricou and Romaric Duvignau

This document is the supplementary material for the paper named above, which points to the sections here that hold its evidence. Timestamps and spans are those of Equation 1 and Fig. 1 of the paper, and both processes read one clock unless a result says otherwise. Table S1 defines the terms used here that the paper does not. The complete record of how the results were obtained, with every result that did not survive, is a separate document in the archived artifact, the postmortem. Pointers such as "postmortem, S37" refer to its sections.

S1. The setup in full

What a reader needs to repeat a run is here: the testbeds and their clocks, every setting that can move a latency number, and where each timestamp is taken.

S1.1. Testbeds, clocks and workload

Two testbeds carry the evidence (Table S2). On the cloud testbed the producer and the consumer run as separate processes on one host with 8 online CPUs, and read one wall clock, CLOCK_REALTIME, through `time.time_ns()`. A wall clock gives every host one epoch, which the cross-host runs need, and on one host its readings came out in order: no latency timed from the publish call falls below zero (S1.3). No scheduler tunable is written anywhere in the archived campaign, which is strong evidence, short of proof, that the kernel's defaults applied, among them the derived base slice of 2.8 ms (Section 4.3). The clocksource was not recorded, but a measured read cost narrows down which one it was (S3.6). Across hosts, `chrony` reports an inter-host offset of 0.067 ms but bounds its own error at 4–7 ms per host over the 4 hosts captured, far more than the offset, so we claim no cross-host bound.

Both testbeds start the brokers with one script, `cloud/brokers.sh`. Kafka 4.1.1, from the image `apache/kafka:4.1.1`, runs as a single node that is also its own controller, the role that keeps the cluster's metadata (KRaft mode). Redis 7, from the image `redis:7`, runs beside it on the same broker machine. On the cloud testbed both brokers ran on one of the three broker machines. In a control of 50 runs for each broker, the *co-located control*, both ran on the client machine itself. The clients are the Python libraries `kafka-python` and `redis-py`, and the registered campaign also ran Kafka's own Java client (S6.1). Table S2 gives each testbed's machines and versions, and Table S3 the settings of the brokers and the clients.

The producer replays StatsBomb's open football event data [S1]. On the cloud testbed it replays eleven of its $3{,}315$ matches, in order, at a chosen publish rate. The registered campaign replays a steady 50 messages a second, except in its block M0, which replays a match's bursts.

G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie.
R. Duvignau is with the Department of Computer Science and Engineering, Chalmers University of Technology, Gothenburg, Sweden. E-mail: duvignau@chalmers.se.

TABLE S1
Terms used here that the paper does not define.

Term	Meaning
cell	A condition together with a workload; where the workload is fixed, the two coincide. A run executes a cell once, and a replicate is a further run of it.
configuration	One setting of a manipulation that compares settings differing in one variable.
core count k	The number of cores the background load occupies.
grid	The instants, τ apart, at which a timestamp changes value.
q-lattice	The retentions $0, 1/q, \ldots, 1$ a paced producer can reach when its publish interval over τ is a/q in lowest terms.

TABLE S2
EACH TESTBED CARRIES A DIFFERENT PART OF THE EVIDENCE. MACHINES, SOFTWARE, AND WHAT RESTS ON EACH TESTBED.

Testbed	Machines	Kernel and software	What rests on it
Cloud	Four Oracle Cloud `VM.Standard.E5.Flex` instances on a private network: one driver, the client machine, with 4 OCPUs, Oracle's unit of two virtual CPUs, and 24 GB of memory, and three broker machines with 1 OCPU and 8 GB each (`cloud/README.md`).	Ubuntu 22.04, kernel `6.8.0-1057-oracle` (EEVDF scheduler), a 1 ms tick; CPython 3.10.12; `kafka-python` and `redis-py`, installed without a version pin when the machines were made in July 2026 (`cloud/cloud-init.yaml`), at versions no record keeps; `chrony`-disciplined clocks, logged every minute.	Every measurement the paper does not attribute to the registered campaign or its machines.
Registered campaign	Three Microsoft Azure pairs in one region, each a driver with eight virtual CPUs and a broker with two: two x86 pairs of `D8as_v6` drivers and `D2as_v6` brokers (AMD EPYC 9V74), and one Arm pair of a `D8ps_v6` and a `D2ps_v6` (Neoverse N2) (`cloud/azure/testbed.json`).	The provider's stock kernels, 6.8.0-1064 and 6.8.0-1065; for the tick block, three kernels built from one source tree that differ in the tick alone, 1, 4 and 10 ms. CPython 3.10.12, with `kafka-python` 2.3.2 and `redis-py` 7.4.1, as read on the drivers on 15 October 2026 (`docs/results/platform/azure_drivers_2026-10-15.md`), under the machine image's pins `kafka-python>=2.3,<3` and `redis>=7,<8` (`cloud/azure/cloud-init.yaml`); Kafka's Java client, `kafka-clients` 3.9.0 (`harness/java/build.sh`), on the drivers' default Java runtime, OpenJDK 11 when read that day.	The registered campaign (S6), with its tools block, and the later runs that time the receiving thread's wait.

S1.2. Every setting that can move a latency number

Table S3 states every broker and client setting that can move a latency number, and why it holds its value.

S1.3. Where each timestamp is taken, and which spans fall below zero

The publish timestamp is read on the calling thread just before the client is invoked. The two clients read the acknowledgment timestamp in different places, but in both it is read only after a thread that was not running has been woken and then given a core. Under Redis it is read on the calling thread the moment the synchronous `xadd` returns, with no second thread involved. Under Kafka it is read, by default, in the client's I/O thread when the delivery callback fires. Our Kafka producer's `inline` mode, which requires one request outstanding, instead timestamps on the calling thread once the publish future resolves.

The consumer reads t_{recv} when it holds the record, and t_{out} once it has handled it. Our Kafka consumer parses the payload inside `poll()`, before t_{recv}, and our Redis consumer parses it between the two. The processing-time latency $t_{\mathrm{out}} - t_{\mathrm{recv}}$ is therefore 290 ns under Kafka and 20,772 ns under Redis, medians of run medians over 5,863 runs, and negative on none of the 708,505 messages. The parse thus falls inside D for Kafka only, and Redis's median end-to-end latency of $824\,\mu$s leaves out a parse 2.5% as long. A longer D puts fewer values of S below zero, and Kafka already has the higher rate, so including that parse would widen the gap between the brokers in Table 1.

Table 1 counts the values below zero in each span over every cloud run that produced matched records, rejected runs included, because a rejected run shows which span falls below zero as well as a clean one does. It leaves out the messages a consumer received late, while it was still reading earlier runs' messages (S1.5). Only S, the one span whose two timestamps have no causal order (Section 4.1), ever does.

By condition the rate varies widely: over the 74 conditions behind the table, each pooling its runs of both verdicts, its median is 1.8% (quartiles 0.8–11.3%), 1.0% on Kafka (0.4–3.5%) and 2.8% on Redis (1.5–12.1%).

S1.4. Which runs each number rests on

The paper's numbers rest on nine bodies of runs, each chosen by its own rule (Table S4). They overlap but do not coincide, so a count from one is not a count from another.

TABLE S3
EVERY BROKER AND CLIENT SETTING THAT CAN MOVE A LATENCY NUMBER, AND WHY IT HOLDS ITS VALUE.

Setting	Value	Why
Brokers		
Kafka nodes	1	One node is both broker and controller, so no write waits for another node.
partitions	1	Every message of a run is written to one log and read by one consumer, in order.
replication factor	1	With one copy, `min.insync.replicas` has no effect, and `acks=all` waits only for the broker's own append.
Redis persistence	off	No snapshot and no append-only file, so `XADD` returns once the entry is in memory.
Kafka producer		
`acks`	`all`	Durability matched to Redis's synchronous `XADD`.
`linger.ms`	0	No artificial batching delay: on a sparse workload any linger would dominate the quantity measured.
`max-inflight`	64	At our producer's default of 1 the client blocks on a round trip per message.
`compression`	none	Football events are small JSON. Compression would add CPU work to the timed path for no representative benefit.
`ack-stamp`	`callback`	The default, asymmetric with Redis (S1.3).
Redis producer		
publish workers	pool	Non-blocking dispatch keeps the publish loop on schedule; publish and acknowledgment are timestamped in the worker.
Consumers		
Redis read	group	The consumer reads through a consumer group, with `XREADGROUP`, because only what is read through a group can be acknowledged.
Redis `ack-batch`	200	Acknowledging each message separately adds a round trip per message.
Redis `count`	200	Read batch, matched to the acknowledgment batch so neither dominates.
Kafka `poll-timeout`	1000 ms	The client already amortizes fetches. No per-record round trip exists to remove.
Replay		
speed-up	derived	Plans carry a built-in 120× compression, so the flag is not the rate. The rate is derived from the plan and checked against wall time before every campaign.

TABLE S4
EACH NUMBER RESTS ON A STATED BODY OF RUNS. WHERE A NUMBER COMES FROM, HOW ITS RUNS WERE CHOSEN, AND WHERE THE PAPER AND THIS SUPPLEMENT USE IT.

Numbers	Runs and messages	How the runs were chosen	Where
The distribution of S, 8.79% below zero; the message size and publish rate, from the same runs' producer logs	5,863 runs, 708,505 messages	every cloud run with matched records, rejected or not	II, III; S1.3
The sign check	884 runs in 40 conditions	the runs behind a reported result	VI; S2
The understatement, the recovery and the exposure band	70 groups from 51 conditions, 665,299 messages	each condition's runs, split by the sign check's verdict	VI; S2.1
The manipulations	25 runs and 2,408–2,985 messages a configuration, on each broker	the campaigns E-A3 to E-A10	IV-B; S3
The traced waits	551,956 wakeups	E-A9, normal priority, 88% load	IV-C; S3.7
The positivity filter	75 settings with a captured summary	the benchmark, embedded mode	V-C; S4
The public reports	43 reports	the registered audit's search and rule	V-C; S5
The registered campaign	8,102 runs in 156 campaigns, and 1,516,342 acknowledgments recorded	three Azure pairs, a plan frozen before its runs	IV, VIII; S6
The kernel-timed latency and the acknowledgment's arrival	48, 48 and 156 runs in freezes 39, 40 and 41	blocks fixed before their runs, on the two x86 pairs	IV-A, VI, VIII; S3.9–S3.11

S1.5. Two faults of the cloud testbed, and the messages left out

Two faults of the cloud testbed's consumers came to light after its runs, when the testbed was gone and no run could be repeated. One lost messages. The other delayed them, and it decides which messages every number from that testbed counts.

The consumer stopped during long pauses: A consumer stops after 15 s without a message. The producer replays a match in real time, and many matches pause for longer. In 1,491 runs the consumer stopped during such a pause, and the producer published the rest of the match to a broker that acknowledged it and that no one read: 91,177 of the 829,907 messages published, each among its run's last. A message no one received has no end-to-end latency, so no number counts these.

The consumer first read earlier runs' messages: The runs reused topic and stream names from one campaign to the next, and nothing cleared them. The Redis trial deleted its stream with a command-line client the driver did not have, and discarded the error; the Kafka trial cleared nothing. A run's consumer reads its topic or stream from the beginning and discards other

runs' messages. So it first read every earlier campaign's, a median of 6,010 messages on Redis, whose consumer logs each read, and meanwhile the run's own first messages waited in the broker. A message that waited there was received long after its acknowledgment. It cannot put S below zero, so it dilutes the rate.

The rule: A message is late when its delivery took more than a second longer than the run's fastest. The consumer caught up when it received the last such message. A message published before then that arrived within a second of that instant came in the same burst, so it is late too, although it waited less. Every message of a run is late when even its fastest delivery took over a second, because its consumer never caught up. The second is a wide margin. A run's fastest delivery took 1.5 ms at the median and under 111 ms in 99% of runs, and over a second only in the 50 runs whose consumer never caught up.

How the rule was checked: None of the 30,225 messages it finds has a value of S below zero. In every one of the 2,099 runs that have any, the run's first published message is one of them, so each backlog began with its run. Every Redis run with a late message whose consumer logged its reads had read other runs' messages first. The late messages are 2.2% of Kafka's matched messages and 6.0% of Redis's.

What leaving them out changes: Every analysis of the cloud testbed leaves the late messages out (`scripts/stale_backlog.py`). The corpus of Table 1 falls from 738,730 matched messages to 708,505, and its negative-span rate rises from 8.43% to 8.79%. The campaigns whose consumers read the longest backlogs move most. The padding repeat E-A10b had sat below E-A10 and now agrees with it at every padding (Table S9). The traced campaigns' rates rise to their untraced twin's, which removes a finding: the tracer had seemed to lower Kafka's rate at normal priority, and it does not (S3.5). The cells of a table now hold different numbers of messages, and each caption gives the range.

Shorter waits: A backlog the consumer read in under a second escapes the rule, which needs a wait of a second to see one. Leaving out also each run's leading messages whose S exceeds the run's median by more than 300 ms, with those published before the last of them arrived, takes out 1,898 more messages, none below zero. The corpus rate then reads 8.81%, so the margin's choice moves it by hundredths of a point.

What cannot be recomputed: The sign check's counts in S2 come from runs that are no longer in the archive, and were made with the late messages in. A late message can only dilute its run's values below zero, so the check's rejections are a lower bound. The registered campaign deleted its streams and topics before each run. By the same rule 0.06% of its messages are late, and only 31 of its 9,055 runs began with a wait, so its readings stand as judged (S6).

S2. The sign check, and the runs it covers

The rule is that of Section 6, applied to every run. A run is rejected if more than 1% of its messages carry a negative value in S, in the publish delay $t_{\mathrm{pub}} - t_{\mathrm{sched}}$ or in the processing-time latency $t_{\mathrm{out}} - t_{\mathrm{recv}}$, or if the median of any of the three is negative. A condition is usable only if all of its runs survive. The one-percent line is a convention, and we fixed the rule after earlier campaigns had run, which is weaker than registration.

Over the 884 cloud runs behind our results, the check rejects 459 (51.9%), and 13 of their 40 conditions keep every run. Each build recomputes these counts from the verdicts the check committed, condition by condition, in `docs/results/integrity_by_condition.csv` (`scripts/audit_ledger.py`); the runs themselves are no longer in the archive. The check was made with the late messages of S1.5 in, so leaving those messages out could only add rejections. That corpus is not the one of Table 1, which counts every cloud run that produced matched records, rejected or not, 5,863 of them. Of those, 3,985 exceed the one-percent line on S.

S2.1. The groups of runs behind the paper's Section VI

Section 6's understatement, the recovery's errors and the exposure band are computed over 70 groups of runs, 665,299 messages, one group for each condition and sign-check verdict, from 51 conditions (Table S5). For each group the table gives the medians of D, A and S, the error of S as a latency, A/D, and the error of the recovery that adds the median of A to the median of S, as a share of the median of D. The understatement is one over the median, across a broker's groups, of the median of S over the median of D: 3.8× over Kafka's groups (interquartile range 3.8–4.0×, the least 3.3× and the worst 4.8×), and 6.6× over Redis's (4.8–7.2×, the least 1.7× and the worst 8.9×). The recovery's median error is 0.4% over Kafka's 32 groups, with an upper quartile of 1.3% and a worst of 5.1%, and 14.6% over Redis's 38, with 20% and 35%. Broker by broker, the sign check's verdict makes no detectable difference: the Hodges–Lehmann shift of the error, rejected groups minus passed, is 0.16 points over Kafka's 8 and 24 groups [bootstrap, 95%: −0.17 to 0.44] and 1.4 over Redis's 20 and 18 [bootstrap, 95%: −6.5 to 6.5], and no equivalence is claimed. Pooled, the shift is 3.7 points [bootstrap, 95%: 0.2 to 11.3], an interval that excludes zero, because the rejected groups are mostly Redis's, whose errors are the larger (postmortem, S16.9). The shares of messages whose publish latency exceeds half or all of the end-to-end latency are counted message by message over the corpus of Table 1.

The gap between the brokers in Section 6 is read on *matched workloads*: a sender count, a feed count and a sign-check verdict for which both brokers have a group, so that each pair compares one workload on runs the check treated alike. There are 24 such pairs (Table S6). In each, the gap is Kafka's median over Redis's, once of D and once of S, and the distortion

TABLE S5
SECTION VI'S NUMBERS COME FROM 70 GROUPS OF RUNS. GROUP: BROKER (K, R), SENDER COUNT, FEED COUNT, AND THE SIGN CHECK'S VERDICT ON THE GROUP'S RUNS (P PASSED, F REJECTED). D, A AND S: THE MEDIANS OF THE GROUP'S MESSAGES, AT THEIR OWN RESOLUTION, IN MILLISECONDS. A/D: THE ERROR OF S AS A LATENCY, IN PERCENT. REPAIR: THE ERROR OF THE RECOVERY THAT ADDS THE MEDIAN OF A TO THE MEDIAN OF S, IN PERCENT OF THE MEDIAN OF D. GENERATED AT BUILD TIME FROM THE COMMITTED TABLE.

Group	D	A	S	A/D	Repair	Group	D	A	S	A/D	Repair
K 12 1 p	2.06	1.53	0.53	74	0	R 12 12 f	0.75	0.53	0.11	71	15
K 12 10 f	2.02	1.47	0.56	73	0	R 12 12 p	0.72	0.52	0.11	72	13
K 12 10 p	1.98	1.45	0.52	73	0	R 12 3 f	0.81	0.54	0.11	67	19
K 12 12 p	1.98	1.46	0.51	74	0	R 12 4 f	0.88	0.54	0.12	62	24
K 12 3 p	2.01	1.48	0.54	73	0	R 12 5 f	0.72	0.53	0.11	74	12
K 12 4 p	2.05	1.49	0.55	73	0	R 12 5 p	0.71	0.51	0.11	71	13
K 12 5 p	2.05	1.52	0.54	74	0	R 12 6 p	0.78	0.51	0.11	66	19
K 12 6 p	2.01	1.49	0.52	74	0	R 12 7 f	1.13	0.53	0.65	47	4
K 12 7 p	2.04	1.49	0.55	73	0	R 12 8 f	1.05	0.52	0.21	50	31
K 12 8 f	2.04	1.50	0.55	73	0	R 12 9 f	0.75	0.53	0.11	70	16
K 12 9 p	2.02	1.50	0.53	74	1	R 12 9 p	0.72	0.53	0.10	74	13
K 1 1 p	2.18	1.70	0.50	78	1	R 1 1 f	0.74	0.56	0.09	75	13
K 2 1 p	1.77	1.31	0.37	74	5	R 1 1 p	0.72	0.57	0.08	79	9
K 2 2 p	1.62	1.23	0.36	76	2	R 2 1 f	0.82	0.56	0.10	68	20
K 5 1 f	2.90	2.21	0.70	76	1	R 2 1 p	0.75	0.55	0.12	73	11
K 5 1 p	2.69	1.91	0.68	71	4	R 5 1 f	1.21	0.70	0.23	58	23
K 5 2 f	2.97	2.25	0.73	76	1	R 5 1 p	1.24	0.65	0.20	52	31
K 5 2 p	2.66	1.86	0.66	70	5	R 5 2 f	1.24	0.68	0.29	55	22
K 5 3 f	2.87	2.20	0.71	77	1	R 5 2 p	1.23	0.67	0.17	54	32
K 5 3 p	2.62	1.85	0.66	71	4	R 5 3 f	1.23	0.68	0.41	55	12
K 5 4 f	2.93	2.22	0.73	76	0	R 5 3 p	1.20	0.66	0.17	55	31
K 5 4 p	2.74	1.91	0.70	70	5	R 5 4 f	1.28	0.70	0.42	54	13
K 5 5 f	2.76	2.09	0.64	76	1	R 5 4 p	1.30	0.67	0.65	51	1
K 5 5 p	2.82	1.96	0.86	69	0	R 5 5 f	1.15	0.69	0.20	60	23
K 9 1 p	2.06	1.53	0.52	74	0	R 5 5 p	1.23	0.66	0.14	54	35
K 9 3 p	2.03	1.49	0.54	73	0	R 9 1 f	0.83	0.56	0.11	68	19
K 9 4 p	2.10	1.54	0.56	73	0	R 9 1 p	0.77	0.55	0.11	71	14
K 9 5 p	2.06	1.51	0.55	74	0	R 9 3 p	0.74	0.50	0.12	68	16
K 9 6 p	1.98	1.46	0.52	74	0	R 9 4 f	0.83	0.54	0.13	65	19
K 9 7 p	2.04	1.48	0.55	73	0	R 9 5 p	0.70	0.50	0.11	72	13
K 9 8 f	2.00	1.47	0.54	74	1	R 9 6 p	0.77	0.51	0.12	66	18
K 9 9 p	1.95	1.46	0.52	75	1	R 9 7 f	1.11	0.51	0.65	46	4
R 12 1 f	0.81	0.57	0.10	70	17	R 9 8 p	1.07	0.50	0.52	47	5
R 12 10 f	1.09	0.54	0.57	49	1	R 9 9 f	0.72	0.51	0.11	70	15
R 12 10 p	1.05	0.52	0.55	50	2	R 9 9 p	0.71	0.53	0.10	74	12

is the second gap over the first. S inflates the gap in 79% of the pairs and shrinks it in the rest, and the median distortion is $1.7\times$ (interquartile range 1.3–1.8).

Fig. S1 draws the groups against the curve the paper's Section 6 describes. On one path, at a fixed publish latency, the error of S as a latency, A/D, falls as $1/D$. From one condition to the next Kafka's A grows with D, so all 32 of its groups lie above that curve, against 16 of Redis's 38.

S3. THE MECHANISM AND ITS LAWS IN FULL

Codes beginning E-A name campaigns on the Oracle Cloud testbed, and a code ending in b repeats or extends the one before it. Only the receiving thread's wait (S3.8) was measured elsewhere, on the registered campaign's machines.

S3.1. Real-time priority cuts the rate in every matched pair

Moving the threads that read the clock to real-time priority, `SCHED_FIFO`, cut the negative-span rate between 7- and 80-fold on Kafka, and between 6.0- and 7.8-fold on Redis, in each of eight matched pairs, with utilization unchanged (Table S7). At real-time priority those threads preempt the background load instead of queueing behind it, so their waiting probability falls while the machine stays as busy. Any account in which utilization alone sets the rate predicts no change.

The pairs come from three campaigns. E-A5 is the original manipulation, E-A5b widened the range of loads to 60 and 95%, and E-A7 replicated E-A5. A rule fixed in advance withholds a pair whose two achieved utilizations differ by more than 0.05. It withheld 0 pairs, and the widest difference in the table, 0.002, is 25 times smaller than the tolerance.

Only the normal-priority rate moves with load. The real-time rate stays between 0.0017 and 0.0144 over the whole ladder, as it should if priority moves the waiting probability p of Equation 3 directly. The factor tends to grow with load but not at every step, so we report a range. For E-A5's two pairs, resampling whole runs, its 95% intervals are $[15\text{–}\infty]$ at 75% load and $[25\text{–}451]$ at 88%. They are wide because the real-time configurations hold few values of S below zero, in few runs.

Every campaign ran both brokers, 25 runs a configuration on each, and Table S7 gives both; the E-A5 intervals above are Kafka's, the half the first analyses read. Redis's real-time rate settled at 0.0176–0.0255. That floor barely moved with load: its median over the pairs was 0.0191 at 60–75% load and 0.0226 at 88–95%, while the normal-priority rate rose from 0.1229

TABLE S6
ON 79% OF THE 24 MATCHED WORKLOADS, S INFLATES THE GAP BETWEEN THE BROKERS. ONE ROW PER WORKLOAD: SENDER COUNT, FEED COUNT AND THE SIGN CHECK'S VERDICT (P PASSED, F REJECTED); EACH BROKER'S MEDIAN OF D AND OF S, IN MILLISECONDS; THE GAP, KAFKA'S MEDIAN OVER REDIS'S, UNDER D AND UNDER S; AND THE DISTORTION, THE SECOND GAP OVER THE FIRST. GENERATED AT BUILD TIME FROM THE COMMITTED TABLE.

			D (ms)		S (ms)		Gap, Kafka over Redis		
Senders	Feeds	Check	Kafka	Redis	Kafka	Redis	under D	under S	Distortion
1	1	p	2.18	0.72	0.50	0.08	3.04	5.97	1.96
2	1	p	1.77	0.75	0.37	0.12	2.36	3.14	1.34
5	1	f	2.90	1.21	0.70	0.23	2.40	3.01	1.25
5	1	p	2.69	1.24	0.68	0.20	2.18	3.36	1.55
5	2	f	2.97	1.24	0.73	0.29	2.40	2.50	1.04
5	2	p	2.66	1.23	0.66	0.17	2.16	3.79	1.76
5	3	f	2.87	1.23	0.71	0.41	2.32	1.74	0.75
5	3	p	2.62	1.20	0.66	0.17	2.19	3.89	1.78
5	4	f	2.93	1.28	0.73	0.42	2.30	1.73	0.75
5	4	p	2.74	1.30	0.70	0.65	2.10	1.07	0.51
5	5	f	2.76	1.15	0.64	0.20	2.40	3.17	1.32
5	5	p	2.82	1.23	0.86	0.14	2.30	6.22	2.70
9	1	p	2.06	0.77	0.52	0.11	2.68	4.79	1.79
9	3	p	2.03	0.74	0.54	0.12	2.75	4.53	1.65
9	5	p	2.06	0.70	0.55	0.11	2.92	5.04	1.73
9	6	p	1.98	0.77	0.52	0.12	2.58	4.29	1.66
9	9	p	1.95	0.71	0.52	0.10	2.74	5.30	1.93
12	5	p	2.05	0.71	0.54	0.11	2.90	5.03	1.74
12	6	p	2.01	0.78	0.52	0.11	2.58	4.54	1.76
12	8	f	2.04	1.05	0.55	0.21	1.94	2.61	1.35
12	9	p	2.02	0.72	0.53	0.10	2.81	5.51	1.96
12	10	f	2.02	1.09	0.56	0.57	1.85	0.98	0.53
12	10	p	1.98	1.05	0.52	0.55	1.90	0.96	0.50
12	12	p	1.98	0.72	0.51	0.11	2.73	4.74	1.73

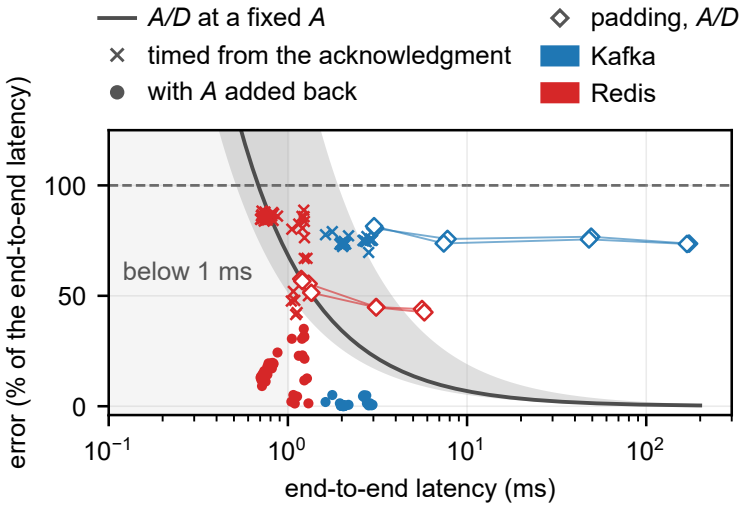


Fig. S1. **Adding the publish latency back cuts the error to 0.4% on Kafka and 14.6% on Redis at the median.** The error of S as a latency on each of the 70 groups of runs, one per condition and sign-check verdict (crosses), and once the publish latency is added back (dots), against the group's end-to-end latency. The curve is A/D for one path at the median publish latency, and its band covers the tenth to ninetieth percentiles of A across groups, 514–1906 μs. Kafka's crosses all sit above it, because its conditions with a longer end-to-end latency also have a longer publish latency, and Redis's straddle it. On Kafka the padded messages' groups (diamonds) stay high too, since their publish latency grows with the payload almost as fast as D. On the dashed line A equals D. The shaded decade holds the sub-millisecond medians now reported for brokers.

to 0.1676. Unlike Kafka's, which sits among the first five messages of a run, published together at its first instant and waiting 107–395 ms for the first acknowledgment, it is spread through the run, at ranks 3 to 145 of its runs' messages. Timed by the kernel in freeze 41, all but a few of Redis's real-time values below zero had the delivery first: that floor is the broker's reply order (S3.11). Intervals that treat messages as independent, Wilson's on each rate [S2] and Katz's on their ratio [S3], are too narrow here, since values below zero cluster within a run (S3.4), so the intervals above and the paper's resample whole runs instead. On those, every pair's two rates stay apart on both brokers. The smallest lower bound over the pairs, the paper's at-least factor, is 4.4 on Kafka, from E-A5b's 70% pair, and 4.2 on Redis, from E-A5b's 70% pair. On Kafka five pairs' factors have no upper bound, against 0 on Redis, because their real-time values below zero sit in a few of the 25 runs, and a resample

TABLE S7

FACTOR OF 7 TO 80 ON KAFKA, WITH UTILIZATION MATCHED IN EVERY PAIR. NORMAL-PRIORITY AND REAL-TIME NEGATIVE-SPAN RATES AT THE SAME ACHIEVED UTILIZATION, AND THE FACTOR BETWEEN THEM, ON BOTH BROKERS, FROM THREE CAMPAIGNS, WITH A 95% INTERVAL THAT RESAMPLES WHOLE RUNS IN BRACKETS, ROUNDED OUTWARD (∞: NO UPPER END). EVERY CELL IS 2,822 TO 2,985 MATCHED EVENTS. BOTH CONFIGURATIONS' ACHIEVED ρ IS PRINTED SO THAT THE MANIPULATION CAN BE CHECKED PAIR BY PAIR. GENERATED AT BUILD TIME FROM THE CAMPAIGN FILES.

				Kafka			Redis		
campaign	load	ρ normal	ρ real-time	normal	real-time	factor	normal	real-time	factor
E-A5b	60%	0.6055	0.6035	0.0459	0.0017	27.4 [8.1–∞]	0.1117	0.0186	6.0 [4.6–8.2]
E-A5b	70%	0.7032	0.7025	0.1032	0.0144	7.2 [4.4–14.8]	0.1174	0.0197	6.0 [4.2–9.2]
E-A5	75%	0.7531	0.7525	0.1320	0.0034	39.4 [15.1–∞]	0.1284	0.0202	6.3 [5.0–8.3]
E-A7	75%	0.7550	0.7537	0.1504	0.0044	34.5 [15.4–∞]	0.1301	0.0176	7.4 [5.6–10.0]
E-A5b	88%	0.8806	0.8809	0.2223	0.0034	66.4 [26.2–∞]	0.1636	0.0220	7.4 [5.3–11.2]
E-A5	88%	0.8809	0.8812	0.3049	0.0057	53.5 [24.6–451.5]	0.1666	0.0255	6.5 [5.1–8.8]
E-A7	88%	0.8825	0.8825	0.2532	0.0054	47.2 [22.0–685.9]	0.1687	0.0226	7.5 [5.6–10.2]
E-A5b	95%	0.9501	0.9501	0.2943	0.0037	79.9 [34.0–∞]	0.1754	0.0225	7.8 [5.7–11.2]

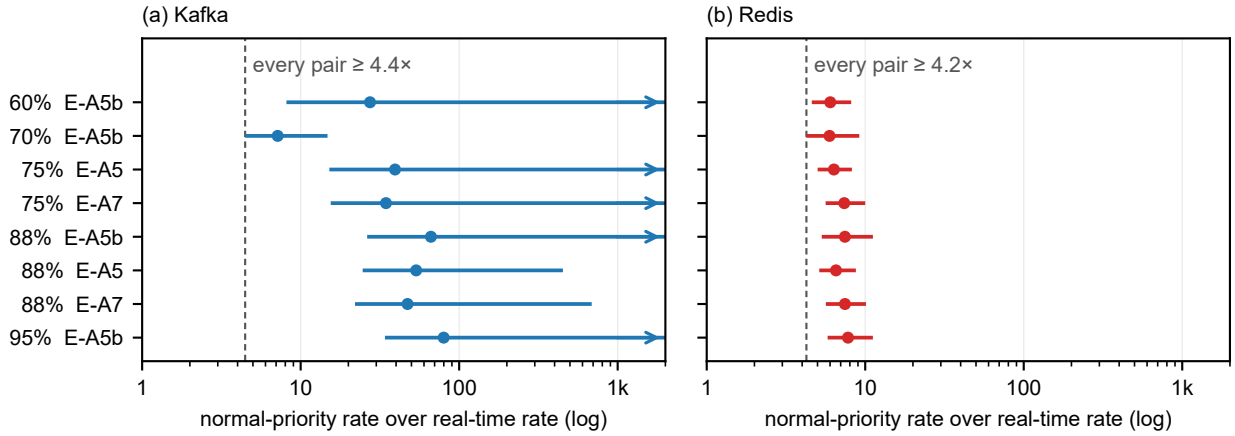


Fig. S2. **Real-time priority cuts the rate in every matched pair, on both brokers.** Each pair's normal-priority rate over its real-time rate, by load and campaign, with a 95% interval that resamples whole runs; an arrow marks an interval with no upper end. The dashed line is the smallest lower bound over the pairs, the paper's at-least factor.

can miss them all. Fig. S2 draws each pair's factor and interval.

E-A7 also registered that real-time priority would cut the thread's waiting probability at least tenfold. Its scheduler counters show the waiting probability falling 1.7- and 2.2-fold at its two loads, and the mean wait 3- to 5-fold, against the falls in the rate that Table S7 gives for E-A7. The prediction failed. Counters that report means cannot see a tail event, so these campaigns leave the factorization of Equation 3 untested. The kernel trace of S3.5 times the wait directly.

S3.2. One utilization, placed two ways, gives two rates

A concentrated load keeps k of the $C = 8$ cores fully busy and leaves the other $C - k$ free, while a spread load keeps every core busy part of the time and leaves none free. If values of S below zero come from a thread that cannot find a free core, the spread load should produce more of them at equal utilization, and on Kafka it does (Table S8, Kafka's runs).

At $k = 6$ both placements sit at $\rho = 0.7531$, identical to four decimals, on one machine in one hour, and their rates differ 2.07-fold ($z = 10.3$ if messages were independent; bootstrap over runs, 95%: 1.36–3.60). No function of ρ can do that, since a function returns one value for one input. The replication E-A6b reproduces the cell at the same ρ and a factor of 2.02 [bootstrap over runs, 95%: 1.33–3.49]. At $k = 5$ both campaigns keep the direction once whole runs are resampled [bootstrap over runs, 95%: 1.10–3.64 in the original, 2.58–9.28 in the replication]. At $k = 7$ the difference is small, since one free core in eight is nearly as useless as none, and neither run resolves it once whole runs are resampled: both intervals take in no difference at all [bootstrap over runs, 95%: 0.93–1.21 in the original, 0.99–1.39 around the replication's 1.17]. Multi-server queueing already expects placement to matter at fixed ρ, so the contrast refutes not queueing itself but the single-parameter form, in which ρ alone fixes the rate.

On Redis, in the same runs, the order reverses. At $k = 6$ the spread load's rate is 0.65 times the concentrated load's [bootstrap over runs, 95%: 0.55–0.78], and 0.62 times in the replication [bootstrap, 95%: 0.51–0.76]. One utilization still gives two rates, which is what refutes a rate set by utilization alone, but the explanation above fits Kafka only. We have not tested why Redis's clients fare worse with two cores free. One candidate is its producer's pool of worker threads (Table S3), which may need more cores than a concentrated load leaves.

TABLE S8
AT $k = 6$ ONE UTILIZATION GIVES TWO RATES, IN TWO CAMPAIGNS. CONCENTRATED LOAD LEAVES $C - k$ CORES FREE; SPREAD LOAD DUTY-CYCLES ALL OF THEM AND LEAVES NONE. EVERY CELL IS KAFKA'S 2,497 TO 2,985 MATCHED EVENTS OVER 25 RUNS, AND z IS A TWO-PROPORTION TEST BETWEEN THE TWO PLACEMENTS OF A ROW, WHICH TREATS MESSAGES AS INDEPENDENT. GENERATED AT BUILD TIME FROM THE CAMPAIGN FILES. ACHIEVED ρ AT $k = 6$ IS 0.7531 IN ALL FOUR CONFIGURATIONS; AT $k = 5$ IT IS 0.6284/0.6250 AND 0.6284/0.6259, AND AT $k = 7$ 0.8778/0.8809 AND 0.8778/0.8822.

	E-A6 (original)			E-A6b (replication)			
k/C	concentrated	spread	z	concentrated	spread	z	factors
5/8	0.0201	0.0379	4.09	0.0128	0.0584	8.83	1.88×, 4.55×
6/8	0.0824	0.1709	10.27	0.0706	0.1429	8.34	2.07×, 2.02×
7/8	0.2281	0.2415	1.22	0.2359	0.2752	3.20	1.06×, 1.17×

TABLE S9
SLOWER PATH, MORE RELIABLE MEASUREMENT, IN TWO CAMPAIGNS. PAYLOAD PADDING IN TWO CAMPAIGNS. THE MEDIAN OF S SPANS 74.3× AND 77.3× WHILE THE RATE FALLS 3.85× AND 4.04×, BOTH MONOTONICALLY, WITH UTILIZATION HELD TO THE ρ SPAN SHOWN. EACH CELL IS 2,408 TO 2,980 EVENTS. GENERATED AT BUILD TIME FROM THE CAMPAIGN FILES.

	E-A10		E-A10b		
Padding (B)	Median S (ms)	Rate	Median S (ms)	Rate	ρ span
0	0.697	0.2627	0.670	0.2749	0.881–0.882
4,096	2.378	0.1909	2.199	0.1944	0.882–0.882
65,536	14.469	0.0901	13.973	0.0928	0.885–0.885
262,144	51.768	0.0683	51.795	0.0681	0.893–0.895

S3.3. A longer path gives fewer values below zero

The same waits must outlast a longer interval to put S below zero, so a longer path should give fewer such values. Padding the payload lengthens the time to the consumer's reading, by transfer and parsing, at nearly unchanged utilization. In E-A10 the median of S rose 74.3-fold over 4 levels while Kafka's negative-span rate fell 3.8-fold [bootstrap over runs, 95%: 3.1–4.8], monotonically, and a repeat on another day, E-A10b, agrees (4.04-fold [bootstrap, 95%: 3.32–5.01]; Table S9). On Redis the rate fell 7.9-fold [bootstrap over runs, 95%: 6.2–10.4], and 5.7-fold in the repeat, though not at every step there. Serialization pushes utilization slightly up, which would raise the rate, so the fall is measured against its own confound. An account in which stress on the host causes the values below zero predicts the opposite direction.

S3.4. Load mostly changes how often the thread waits

A thread that reads the clock is either running, and reads it late by a narrow jitter, or waiting for a core. From an idle host to the heaviest load before its processors are fully busy, E-A3's negative-span rate grows 61-fold while the jitter's width grows only 5-fold (Table S10). Load therefore moves mostly the waiting probability p of Equation 3, the chance the thread is waiting, and little the running state's jitter. A scale family, in which load only widens one distribution, has one parameter and cannot move the two this differently. The tails agree. If load moves only p, the tail curves of two loads should shift vertically and stay parallel on a log scale, where a scale family would rescale them horizontally. Over tail estimates from at least 20 events, the log ratio of two conditions' tail masses varies over thresholds by a median factor of 1.34 across conditions, while the scale-family collapse fails by 23×. That test was exploratory; the registered test of p is the priority manipulation of S3.1.

A wait lasts an interval, so the messages inside one are read late together. A Wald–Wolfowitz runs test on each run's sequence of signs of S finds strong clustering at every load, even with no background load. Coarse kernel timestamps would corrupt messages independently and give $z \approx 0$.

Equation 3 describes how the rate moves between conditions, not single messages. Read with the measured D in place of the margin, it would have the rate fall as D grows inside a condition, and it does not. We counted each condition's messages in bins of D, 8 to a decade with at least 172 messages a bin, a count fixed before any bin was read. The rate rises from one bin to a longer one, with disjoint Wilson intervals [S2], in 24 of 56 testable conditions, and in 21 to 29 over four, eight and twelve bins a decade. The rises sit in the runs the sign check rejects (19 of 40 conditions), not in those it passes (2 of 50), whose rates are low. Within a run D and A correlate, with a median coefficient of 0.80 over our groups, so a long delivery comes with a long publish latency, and a message chosen for its long D likely had its acknowledgment read late too. These runs could not read the margin message by message; freeze 41 later did, as S_k, and the rate did not fall from one fifth of it to the next (S3.11). Between conditions, where padding lengthens the margin at fixed load, the rate falls as predicted (S3.3).

S3.5. A kernel trace agrees with the rate, unfitted, and changes it

The negative-span rate should be close to the probability that a wait for a core outlasts the margin, the median of S with no load, and the margin differs by broker: in E-A3's idle runs it is 0.52 ms on Kafka and 0.10 ms on Redis (`docs/results/`

TABLE S10
SHORT OF FULL UTILIZATION THE RATE GROWS 61-FOLD AND THE JITTER ONLY 5-FOLD. E-A3, NINE LOAD LEVELS, 25 RUNS EACH, REGISTERED. THE JITTER WIDTH IS $\sigma_{\text{core}} = \text{IQR}/1.349$. VALUES OF S BELOW ZERO CLUSTER AT EVERY LOAD (RUNS-TEST z). THE THREE $\rho = 1$ ROWS ARE DISTINCT OVERSUBSCRIPTION LEVELS THAT THE UTILIZATION CANNOT SEPARATE.

Utilization ρ	Jitter width σ_{core} (ms)	Negative-span rate	Clustering z
0.003	0.126	0.004	−6.8
0.253	0.138	0.004	−6.8
0.504	0.180	0.006	−6.8
0.628	0.181	0.024	−6.9
0.753	0.238	0.076	−5.7
0.878	0.631	0.224	−4.9
1.000	1.224	0.371	−4.3
1.000	1.293	0.317	−4.7
1.000	1.710	0.261	−4.4

TABLE S11
POOLED, THE TRACE STAYS WITHIN A FACTOR OF THREE OF THE RATE AT EVERY THRESHOLD; AT REDIS'S OWN MARGIN IT REACHES 3.3×. THE SHARE OF TRACED WAITS FOR A CORE LONGER THAN EACH THRESHOLD OVER THE NEGATIVE-SPAN RATE OF THE SAME RUNS, AT NORMAL PRIORITY: BOTH BROKERS' THREADS AGAINST THE POOLED RATE, THEN EACH BROKER AT ITS OWN MARGIN m AGAINST ITS OWN RATE. THE TRACE'S BUCKETS ARE POWERS OF TWO IN MICROSECONDS. GENERATED AT BUILD TIME FROM THE TRACE HISTOGRAMS AND THE CAMPAIGN FILES.

Traced share of waits, over the rate	E-A9, 88%	E-A9b, 75%	E-A9b, 88%
Both brokers' threads, against the pooled rate			
Pr[wait ≥ 0.256 ms][a]	1.27	1.61	1.33
Pr[wait > 0.5 ms]	0.84	1.17	1.01
Pr[wait > 1 ms]	0.74	1.01	0.89
Pr[wait > 2 ms]	0.63	0.86	0.78
Each broker at its own margin, against its own rate			
Kafka, $m = 0.52$ ms: Pr[wait > 0.5 ms]	0.66	1.02	0.80
Redis, $m = 0.10$ ms: the buckets from 64 and 128 μs[b]	2.5–3.2	2.5–3.3	2.3–3.0

[a] The bucket edge above 0.25 ms; its share bounds the share over 0.25 ms from below.
[b] The margin falls inside a bucket: from the share at or above 128 μs to the share at or above 64 μs.

`model/broker_margins.csv`). With `bpftrace` on the `sched_wakeup` and `sched_switch` tracepoints we recorded the delay between a thread that reads the clock becoming runnable and reaching a core, for every such transition in a run. The two quantities share no probe, no estimator and no data. One is a count of values below zero in application records, the other a kernel histogram of scheduler transitions. The trace counts every `python3` thread in a configuration, the producers and consumers of both brokers, so we set its share of waits longer than 0.5 ms against the rate of the same runs with both brokers pooled. That threshold, Kafka's margin rounded, was written into `scripts/analyze_stall_duration.py` on 25 July 2026, before the trace ran. Across the three normal-priority configurations of Table S12 the ratio of traced share to that rate is 0.84, 1.01 and 1.17, with nothing fitted. The ratios lie above and below one, so no bias smaller than their spread can be resolved. A rule written into `scripts/analyze_runq_tail.py` before the data counted agreement within a factor of three as a match, and at every threshold from 0.25 to 2 ms the ratio stays within 0.63–1.61 (Table S11).

Read broker by broker, each at its own margin and against its own rate, the trace agrees less well. Against Kafka's rate alone, as first computed, the ratios are 0.66, 0.80, 1.02 at 0.5 ms, and Redis's share at 0.10 ms stands at 2.3–3.3 times Redis's rate. The trace cannot tell the brokers' threads apart, so a pooled ratio near one can come from errors that offset. The check is therefore consistent with the mechanism rather than a test of it; the acknowledgments' own waits, timed in the registered campaign (S6.2), and the kernel-timed order of freeze 41 (S3.11) test it message by message.

A first attempt at the traced run, whose probe failed to attach, became an untraced twin. It ran two hours apart with the same design. At normal priority it measured Kafka's rate at 0.273 against the traced 0.272 ($z = 0.1$), and the pooled rates agree as closely, so there the tracer leaves the rate alone. Each prediction is held against the traced run's own rate all the same, so that a common perturbation would move both quantities together. A rule fixed in advance, the tracer check, compares a traced configuration only if its rate lies within 25% of its untraced comparator. The largest drift is 4% on the pooled rate and 5% on Kafka's, so all three are compared.

At real-time priority the tracer does change the rate, and only Kafka's. Kafka's traced real-time configurations read exactly zero where its untraced control reads 15 in 2,985, while Redis's read 1.6–2.2%, close to its untraced real-time rate. The pooled real-time rate is therefore Redis's halved, and the traced probability stands to it at 1.6–2.0. There the tracer silenced half of what the trace is compared with, so these ratios are shown and not used.

TABLE S12
KERNEL TRACING AGREES WITH THE POOLED NEGATIVE-SPAN RATE, UNFITTED, IN EVERY NORMAL-PRIORITY CONFIGURATION.
$\Pr[\text{WAIT} > 0.5\,\text{MS}]$: THE SHARE OF TRACED WAKEUPS THAT WAITED LONGER, FROM THE `SCHED_WAKEUP` AND `SCHED_SWITCH` TRACEPOINTS. RATE: THE NEGATIVE-SPAN RATE OF THE SAME RUNS, BOTH BROKERS POOLED OR KAFKA'S ALONE. RATIO: THE TRACED SHARE OVER THAT RATE. EACH BROKER CONTRIBUTES 2,257 TO 2,985 MESSAGES A CONFIGURATION.

						Both brokers		Kafka alone	
Campaign	Priority	Load	ρ	Traced wakeups	$\Pr[\text{wait} > 0.5\,\text{ms}]$	Rate	Ratio	Rate	Ratio
E-A9	normal	88%	0.8822	551,956	0.181	0.214	0.84	0.272	0.66
E-A9	real-time	88%	0.8819	570,591	0.018	0.011	1.63‡	0.000	—
E-A9b	normal	75%	0.7543	709,819	0.168	0.144	1.17	0.164	1.02
E-A9b	real-time	75%	0.7531	641,308	0.016	0.008	1.95‡	0.000	—
E-A9b	normal	88%	0.8825	687,986	0.207	0.205	1.01	0.259	0.80
E-A9b	real-time	88%	0.8822	649,788	0.016	0.009	1.71‡	0.000	—
Untraced control: E-A9's first attempt, probe never attached									
—	normal	88%	0.8812	—	—	0.214	—	0.273	—
—	real-time	88%	0.8809	—	—	0.013	—	0.005	—

‡Kafka's half of the pooled rate reads zero under the tracer: shown, not used.

S3.6. Rival explanations: excluded, bounded, or real on Redis

Five rivals fall to the design and the manipulations. One clock excludes clock skew between hosts, the cause a concurrent study assigns to such values (postmortem, S33). One utilization at rates twofold apart excludes utilization (S3.2). Real-time priority excludes the timer noise that reaches a thread whatever its scheduling class, such as interrupts (S3.1), and a longer path that lowers the rate excludes stress on the host (S3.3). Coarse kernel timestamps fall to the runs test of S3.4.

One kind of timer delay does depend on the class, so priority alone cannot exclude it. A thread that sleeps with a timeout may wake later than the timeout by its *timer slack*, the margin the kernel allows itself in order to batch wakeups. In Linux 6.8 a real-time task gets no slack in `select`, `poll`, `epoll_wait` and `nanosleep`, while a normal task gets at least $50\,\mu$s by default. But slack can delay only a wakeup that a timer makes. A thread that reads the clock after arriving data is woken by the data, not by a timer, so slack cannot delay it. Five more rivals need an argument each. Two, the hypervisor and the interpreter lock, are bounded rather than excluded, and a third, the broker's reply order, is excluded on Kafka and real on Redis.

The second thread: Kafka's client reads the clock for the acknowledgment on its I/O thread, and Redis's on the calling thread (Figure 1; S1.3). The hand-off is not the mechanism. Redis, which has no second thread, has values of S below zero on 8.71% of its messages against Kafka's 8.87%, and on more of its runs (90% against 65%). Its calling thread, returning from a blocking socket read, must also be woken and given a core before it reads the clock.

The hypervisor: Both testbeds are virtual machines, so steal time, the time a virtual processor is ready to run but waits while the hypervisor runs another machine, is bounded, not excluded. Two observations disagree with steal time as the cause. Real-time priority is a setting inside the virtual machine, which steal time would not yield to, yet it cuts the rate 7- to 80-fold on Kafka and 6.0- to 7.8-fold on Redis (S3.1). And the upper mode of the traced waits shares its bucket with the base slice that the virtual machine's own kernel derives (S3.7), not with any quantity the hypervisor exposes. The Oracle runs did not record steal time. The registered campaign recorded it from `/proc/stat` in 8,092 runs and found none, but we did not establish whether the Azure hypervisor reports steal time to its machines, so that reading does not exclude it either.

The interpreter lock: The threads that read the clock are CPython, which releases the lock around blocking I/O and retakes it on return, so a timestamp waits for a core and then for the lock. The kernel trace cannot see the second wait, and neither priority nor the size of the waits tells the two apart. The direction of the trace's error bounds that second wait. A wait in series with the run queue would make the trace underestimate the rate, but its ratios to the rate of the runs it traced, 0.84, 1.01 and 1.17, lie on both sides of one. The registered campaign also found values of S below zero in Kafka's own Java client, which has no interpreter lock, at a third to a quarter of the Python client's rate (S6.2). So the effect needs no interpreter lock. The Java result does not show that the lock adds nothing to the Python client's rate. Outside Python too, another concurrent study splits the latency of a Linux host's network stack into parts timed by the kernel. It finds that the main part of the tail is the wait from a thread's wakeup by arriving data to its running [S4].

The clock: Clocks that disagree across virtual CPUs are excluded twice on the Oracle machines, whose hypervisor is KVM [S5]. First, the platform probe's smallest observed increment of `clock_gettime(CLOCK_MONOTONIC)` was 90 ns, an upper bound on the cost of one read. That excludes `acpi_pm` by arithmetic, since its tick is 279 ns, and `hpet` by its published read cost near $1.2\,\mu$s. What remains, `kvm-clock` or `tsc`, carries the coherence guarantee. A `kvm-clock` that fast is on the vDSO path, which `kvmclock.c` enables only when the hypervisor sets `PVCLOCK_TSC_STABLE_BIT`, the flag that guarantees monotonic time across CPUs. A KVM guest selects `tsc` only when the kernel finds it constant, non-stop and stable, and abandons it if it later falls out of step between CPUs. Which of the two ran is unrecorded, and the published read costs vary about twofold, against an order-of-magnitude margin. Second, an offset between clocks large enough to produce

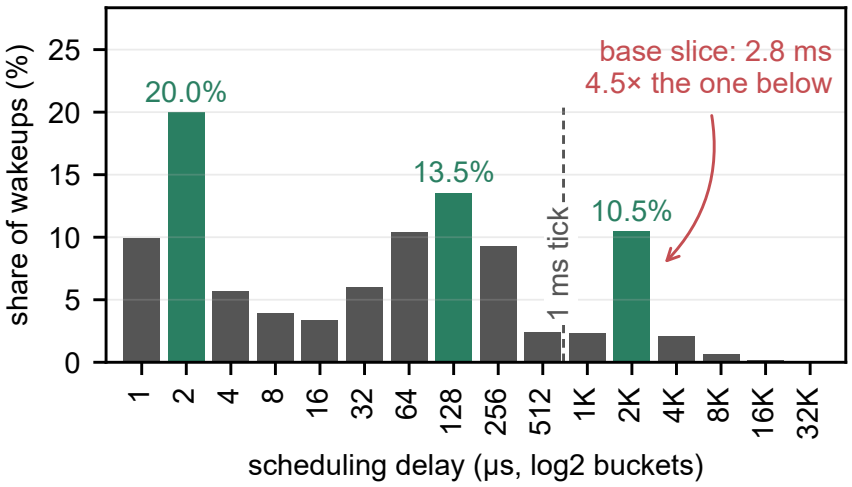


Fig. S3. **Traced scheduling delays are trimodal.** 551,956 wakeups of E-A9's normal-priority configuration at 88% load (Table S12), `bpftrace` $\log_2$ buckets, modes shaded. The jitter peaks in the 2–4 μs bucket, which alone holds 20.0% of all wakeups. A second mode at 128–256 μs holds 13.5%. The third, at 10.5%, sits in the bucket holding the scheduler's derived base slice of 2.8 ms. The count of modes survives a 2× widening of the buckets in both phasings.

the deepest value of S, -99.8 ms, would have driven D far below zero. The smallest D observed, 220 μs, bounds any such offset 455-fold below what that value needs.

Two observations do the same work on Azure, where the registered campaign and the runs of S3.8 to S3.10 ran, under the Hyper-V hypervisor. First, an x86 guest there reads the kernel's clocksource built on the processor's timestamp counter (TSC). Where Hyper-V cannot keep that counter's frequency across a move to another host, the guest reads Hyper-V's own clock instead. It computes that clock from the TSC with a scale and an offset that the hypervisor keeps in a page of memory shared with the guest (`hyperv_clocksource_tsc_page`) [S6]. Either way every CPU turns the counter into time with the same scale and offset, and Hyper-V gives the guest the counter's frequency. An Arm guest reads the system counter that Hyper-V virtualizes for it, through the same driver it would use without a hypervisor. Second, no message of the registered campaign arrived before it was published (S6.2), which bounds any offset between the CPUs' clocks by the shortest end-to-end latency.

The broker's reply order: Both readings follow one write, but the broker then answers twice, with the acknowledgment to the producer and the message to the consumer. A broker that delivered the message first would put S below zero with no wait at all. Freeze 41 timed both arrivals by the kernel (S3.11). On Kafka, which returns the acknowledgment before it answers the consumer's waiting request, the acknowledgment arrived first in all but a few percent of messages, and 99.8% of its values below zero under load had it first: there the rival is excluded. Redis writes the delivery to a waiting consumer before it replies to the producer, and in 16 of its 24 runs the delivery came first for most messages; 95–100% of Redis's values below zero had it first. On Redis the rival is real, and it is the broker's floor at real-time priority. Some brokers deliver before they confirm by design: a persistent RabbitMQ message "can be delivered to the consumer before it's been written to disk" [S7].

S3.7. The waits' upper mode survives coarsening, and its bucket holds the derived slice

The traced waits have 3 local maxima (Figure S3), and the upper one survives a change of bucket edges. How many maxima a histogram shows depends on its bin edges as well as its counts, and these edges came from `bpftrace`. The per-event waits were not retained, so coarsening is the only test. Merging adjacent buckets doubles the bin width, and starting the merge at the first bucket or the second places the edges differently. The spectrum has 3 maxima on the tool's own 16 buckets, 3 again under both phasings at 2 times the width, and fewer at 4 times, so the count holds to within a factor of 2 in bin width.

Buckets that double in width favor the upper ones, so part of the upper mode is the buckets' doing. Waits spread evenly up to the slice and then up to a tick, as Equation S3 has them, would make the bucket that holds the slice 1.2 times the bucket below it. The traced bucket is 4.5 times it, so the waits gather at the slice more than an even spread would put them there. We have not explained the middle mode, at 128–256 μs.

Above the upper mode the counts collapse. They fall 5.0-, 3.4- and 4.7-fold over the next three octaves and then 357-fold, each octave a doubling of the wait, whereas a power law would fall by the same factor in every octave. Power laws fitted below the mode and above it both fail a bootstrap goodness-of-fit ($p < 0.0004$ and $p < 0.0004$, over 2,500 replicates each).

The slice is derived from the kernel's rule and the campaigns' own data. Each placement condition loads k cores and records the achieved utilization, so k/ρ recovers 8 cores six times over, with a spread of 0.05. Equation 4 then gives $4 \times 0.70\,\text{ms} = 2.8$ ms. Upstream Linux 6.8 used $s_0 = 0.75$ ms; the change to 0.70 ms [S8], first released in 6.15, reached this kernel through Ubuntu's stable updates (linux-oracle-6.8 6.8.0-1043, January 2026, with no revert before the 6.8.0-1057 every cloud run

TABLE S13
THREE MEASUREMENTS PUT THE RECEIVING THREAD'S WAIT AT 0.41–0.94 MS OF THE MEAN LATENCY. ONE ROW FOR EACH BROKER AND LOAD. METHOD 1 IS THE FALL IN MEAN D WHEN THE CONSUMER ALONE RUNS AT REAL-TIME PRIORITY. METHODS 2 AND 3 ARE THE FALLS, FROM NORMAL TO REAL-TIME PRIORITY, IN THE MEAN TRACED WAIT AND IN THE MEAN DELAY FROM THE KERNEL'S RECEIVE TIMESTAMP TO THE TIMESTAMP. EACH FIGURE IS A SETUP'S MEDIAN OVER ITS RUNS. THE CLIENT'S WORK IS THE MEDIAN, OVER TRACED RUNS, OF THE DELAY BEYOND THE TRACED WAIT.

	Added to the mean of D (ms)			Largest over	D, normal → real-time priority (ms)		Client's
Broker, load	Method 1	Method 2	Method 3	smallest	Median	Ninetieth percentile	work (ms)
Kafka, 75%	0.43	0.70	0.64	1.64	2.03 → 1.82	5.21 → 4.27	0.37
Kafka, 88%	0.94	0.91	0.92	1.04	2.52 → 1.92	5.66 → 4.53	0.39
Redis, 75%	0.47	0.50	0.47	1.07	0.89 → 0.84	3.06 → 0.93	0.11
Redis, 88%	0.43	0.62	0.41	1.49	0.87 → 0.83	2.86 → 0.89	0.12

recorded), and the registered campaign's Azure kernels, from the same updates, read back the same constant. Either value puts the slice in the 2–4 ms bucket. The build sets `CONFIG_HZ` = 1000, a 1 ms tick. No scheduler tunable is written anywhere in the archived campaigns, so the derived value is the one that applied. That is strong evidence, short of proof, because the slice was not read off the hosts.

A woken thread that the scheduler does not let run at once waits for the running thread to finish its slice, and since a task notices its deadline only at the next tick, the waits end in a band a tick wide. The registered campaign timed the wait itself and found it to be the rest of a slice (S6.3). One rule of the scheduler would explain the wait, *run-to-parity*, which Linux turns on by default (`RUN_TO_PARITY`): under it, `pick_eevdf` in `kernel/sched/fair.c` keeps the running task until it either "becomes non-eligible", having had more than its share of the core, or "gets a new slice", in the words of its comment in Linux 6.8. Freeze 41 turned the rule off (S3.11), and neither the rate of S below zero nor the acknowledging thread's 99th-percentile delay moved on either broker, against a registered cut to two-thirds, so the rule alone is not what holds a woken thread off the core. The rule came with commit 63304558ba5d, "sched/eevdf: Curb wakeup-preemption", first released in Linux 6.6. From Linux 6.17, commit 052c3d87c82e, "sched/fair: Limit run to parity to the min slice of enqueued entities", limits it to the shortest slice queued.

S3.8. Timing from the publish call keeps the receiving thread's wait, under a millisecond on the mean

A latency timed from the publish call carries the receiving thread's wait for a core. By Equation 1, $D = S + A$ for every message, so the acknowledging thread's wait cancels: it lengthens A by as much as it shortens S. The receiving thread's wait does not cancel. The consumer reads its clock only after its thread has been woken and given a core, so every D holds that wait, which is never negative and which the sign check of Section 6 cannot see. After the registered campaign we measured it three ways in the same runs, by a plan and four predictions written before any run.

The runs used the registered campaign's first x86 pair (S6.1), both brokers, 75 and 88% load and the base slice at 3 ms, with no delay added to the network. Eight setups, each broker at each load with the consumer at normal or at real-time priority, ran in six shuffled rounds. Three runs added before anything was read gave every setup at least three traced runs, and all 51 runs passed the integrity rule. Method 1 runs the consumer alone in the real-time class at priority 80 and measures how much that lowers D at unchanged load. Method 2 traces the scheduler in a random half of the runs and measures the receiving thread's own wait for a core before each timestamp, as S3.5 did for the acknowledging thread. Method 3 captures the receiver's packets in every run with the kernel's nanosecond timestamps, and times each message from the packet that carries it to the consumer's timestamp. That delay includes the wake-up, the wait and the client's own work.

The three estimates run from 0.41 to 0.94 ms (Table S13) and lie within a factor of 1.5 of each other at three of the four points. The wait lives in the tail. At normal priority the median message waited 18–46 μs and the mean 0.51–0.92 ms, while the ninetieth percentile reached 2.13–2.92 ms. A concurrent study finds the same kind of wait, outside Python, to be the main part of the tail latency of a Linux host's network stack [S4]. Real-time priority brought the mean wait down to 10–11 μs. On Redis it cut the ninetieth percentile of D to under 1 ms and left the median almost where it was.

The first prediction held. On both brokers at both loads the wait's mean at normal priority lay between 0.3 and 1.2 ms and its ninetieth percentile above 1.5 ms, and real-time priority took the mean below 0.1 ms. The other three failed, each on Kafka. There real-time priority lowered the ninetieth percentile of D by less than the 1 ms predicted at 75% load, and moved the median by more than the 0.3 ms allowed at 88%. Method 3's delay was at least as long as the traced wait for 99.7% or more of every traced run's messages, against the 95% predicted. But Kafka's client then worked longer than the 0.3 ms predicted before its timestamp, where Redis's stayed under it. At 75% load Kafka's first two estimates lie further apart than the factor of 1.5 allows. We have not tested why Kafka differs.

Three things qualify the figures. The tracer is not a free observer: at normal priority the traced runs' mean D ran 5–22% above the untraced runs', so the second method may read the wait somewhat high. In five runs a message arrived more than 150 ms late, three of them during stalls of the whole driver, of the kind the registered campaign marks as paused (S6.2). A setup's value is a median over its runs, which sets them aside. And three of the four setups at real-time priority hold only two

TABLE S14
KERNEL-TIMED LATENCY HELD STILL WHILE, AT NORMAL PRIORITY, THE TAIL OF D ROSE 4.3–4.6-FOLD. W RUNS FROM THE KERNEL'S TIMESTAMP AS A MESSAGE LEAVES THE CLIENT MACHINE TO ITS TIMESTAMP AS THE MESSAGE COMES BACK. D RUNS FROM THE PUBLISH CALL TO THE CONSUMER'S TIMESTAMP. ONE ROW FOR EACH BROKER, BACKGROUND LOAD ON THE CLIENT MACHINE, AND PRIORITY OF THE PRODUCER AND THE CONSUMER TOGETHER. EACH FIGURE IS A SETUP'S MEDIAN OVER ITS SIX RUNS OF EACH RUN'S STATISTIC, IN MICROSECONDS. H_{out} AND H_{in} ARE THE CLIENT MACHINE'S TIME BEFORE AND AFTER W. THE KERNEL'S DELAY, FROM THE NETWORK CARD'S OWN RECEIVE TIMESTAMP TO THE KERNEL'S, IS REPORTED, NOT TESTED AGAINST A PREDICTION.

	W		D		Host's wait, mean		Kernel's
Broker, load, priority	Median	99th	Median	99th	H_{out}	H_{in}	delay
Kafka, 0%, normal	752	822	1684	1801	573	351	16
Kafka, 0%, real-time	744	823	1744	2078	642	372	8
Kafka, 88%, normal	759	838	2443	8299	1199	1247	6
Kafka, 88%, real-time	792	881	1837	2223	663	402	6
Redis, 0%, normal	631	770	851	1011	92	137	20
Redis, 0%, real-time	639	786	834	986	76	129	17
Redis, 88%, normal	613	716	831	4396	88	507	9
Redis, 88%, real-time	588	729	813	985	90	114	11

untraced runs, too few to judge the tracer there. The plan, the register of every run and the reading are in the repository under `docs/results/recv_wait`, read by `scripts/recv_wait.py`.

S3.9. Timed between the kernel's timestamps, a message's latency held still across background load and priority

Every remedy of Section 6 leaves some wait inside the latency. Timing from the publish call cancels the wait of the thread that reads the acknowledgment but keeps the receiving thread's (S3.8), and real-time priority shortens both waits without ending them. A fix that keeps every client thread's wait out of the latency has to take both of a message's timestamps where no such thread reads the clock. The kernel does that: it timestamps each packet as it leaves or arrives, without waiting for any program. We tested that fix by a plan and five predictions fixed before any of its runs, including a short pilot that checked the captures.

The runs used the same first x86 pair (S6.1) and both brokers, with no background load on the client machine or with 88%, the base slice at 3 ms and no delay added to the network. The producer and the consumer ran together at normal or at real-time priority. Eight setups ran in six shuffled rounds. All 48 runs passed the integrity rule, and no message arrived more than 150 ms late. Each run captured the producer's packets as they left through the client machine's kernel and the receiver's packets as they arrived, with the kernel's nanosecond timestamps, on the clock the clients read. Each message then carries four times: the publish call, t_{pub}; the kernel's timestamp of the packet that carried it out, t_{tx}; the kernel's timestamp of the packet that brought it in, t_{rx}; and the consumer's timestamp, t_{recv}. The *kernel-timed latency* $W = t_{\mathrm{rx}} - t_{\mathrm{tx}}$ holds the network and the broker and none of the client threads' waits. The client machine's time before and after it, $H_{\mathrm{out}} = t_{\mathrm{tx}} - t_{\mathrm{pub}}$ and $H_{\mathrm{in}} = t_{\mathrm{recv}} - t_{\mathrm{rx}}$, holds the clients' waits and work, so that $D = H_{\mathrm{out}} + W + H_{\mathrm{in}}$ for every message.

All five predictions held (Table S14). The first two asked that W not move with the load or with the priority. The median of W moved by at most 7.9% between the loads at each priority and between the priorities at each load, on both brokers, against the 10% allowed, and its 99th percentile by at most 7.2% against 25%. With six runs a setup, the run-level intervals on those moves reach 12% at the median and 44% at the 99th percentile, beyond the bands, so the bands hold for the point estimates, not for their intervals (`docs/results/so_fix/move_intervals.csv`). The third asked that nothing be lost. None of the $239{,}424$ messages had W at or below zero, and every message appeared in both captures. The fourth asked that the effect the fix keeps out be present in the same runs. At normal priority the load raised the 99th percentile of D 4.6-fold on Kafka and 4.3-fold on Redis, where the plan required 1.5-fold. The mean of D rose too, and the mean of W moved by at most 5.1% of that rise, against the 20% allowed. The fifth placed Kafka's part of the rise before the bytes leave. Under load at normal priority the mean of Kafka's H_{out} rose $626\,\mu$s, where at least $200\,\mu$s was predicted, because its client hands each message to a background thread that writes it to the network after the publish call returns. Redis's, written by the calling thread, moved $4\,\mu$s, within the $100\,\mu$s allowed. H_{in} rose on both brokers, as S3.8 found, and real-time priority undid most of both rises.

The network card's own receive timestamps were captured beside the kernel's and put on the system's clock by sampling the card's clock twice a second. They were reported, not tested against a prediction. Their gap to the kernel's is the part of the client machine's delay that the kernel's timestamp keeps. The kernel timestamped each packet a median of 6–20 μs after the card did, longest with no background load, as the plan expected of processors that must wake to take the interrupt. The plan words this delay as the card's arrival less the kernel's. The delay reported is the kernel's less the card's.

The kernel's receive timestamp can itself be taken late. The kernel takes it as the driver hands a packet to the network stack [S9], normally in its *softirq* processing, the deferred part of the work a packet's interrupt starts. Under heavy load the kernel moves that processing to its `ksoftirqd` threads [S10]. With threaded NAPI, a setting that runs a driver's receive polling in kernel threads of its own, the processing runs in such threads from the start. Either way the kernel's timestamp is then taken by a thread that itself waits for a core, and that wait falls inside W. The card's own receive timestamp is taken

TABLE S15
KERNEL-TIMED LATENCY READ ON THE CLIENTS' SOCKETS MATCHED THE CAPTURES'. ONE ROW FOR EACH BROKER, BACKGROUND LOAD ON THE CLIENT MACHINE, AND PRIORITY OF THE PRODUCER AND THE CONSUMER TOGETHER. W IS THE KERNEL-TIMED LATENCY FROM THE CAPTURES, AND W_{so} THE SAME LATENCY FROM THE SOCKETS. D RUNS FROM THE PUBLISH CALL TO THE CONSUMER'S TIMESTAMP. EACH FIGURE IS A SETUP'S MEDIAN OVER ITS SIX RUNS OF EACH RUN'S STATISTIC, IN MICROSECONDS.

	W		W_{so}		$\lvert W_{\mathrm{so}} - W \rvert$		D
Broker, load, priority	Median	99th	Median	99th	Median	99th	99th
Kafka, 0%, normal	811	966	814	969	2.6	3.8	1977
Kafka, 0%, real-time	747	848	750	851	2.6	3.7	1880
Kafka, 88%, normal	772	922	775	925	3.0	5.9	8371
Kafka, 88%, real-time	779	943	782	945	3.0	5.9	2361
Redis, 0%, normal	592	742	595	744	2.4	3.6	981
Redis, 0%, real-time	581	732	583	735	2.4	3.7	964
Redis, 88%, normal	596	679	599	682	2.6	5.3	4579
Redis, 88%, real-time	585	722	588	725	2.6	5.2	974

before any such thread runs, as measurements that split a host's time from the wire's use [S11]. In these runs, at 50 messages a second on the first x86 pair, W held still under load all the same; on the second pair Redis's rose by a quarter (S3.11).

The largest move of the median of W, $50\,\mu$s on Redis at real-time priority, was a fall under load. Of that fall, $30\,\mu$s lay between the kernel's departure timestamp and the card's arrival timestamp. We have not tested why. A tool can read the kernel's timestamps on its own sockets, without a capture, and S3.10 tests whether they agree with the captures. The plan, the register of every run and the reading are in the repository under `docs/results/full_fix`, read by `scripts/full_fix.py`.

S3.10. Read on the clients' own sockets, the kernel's timestamps agreed with the captures to within microseconds

A benchmark tool would not run a packet capture. It would ask the kernel for its timestamps on its own sockets, through the `SO_TIMESTAMPING` *socket option*, a setting a program asks the kernel for on one of its sockets. We tested that form by a second plan and five predictions fixed before its runs, with S3.9's captures, taken in the same runs, as the reference.

The runs repeated the design of S3.9, with the same pair and both brokers, no background load on the client machine or 88%, and normal or real-time priority for the producer and the consumer together. Eight setups ran in six shuffled rounds, and all 48 runs passed the integrity rule. A small module replaced the socket class in the producer's and the consumer's processes before their client libraries opened a connection. On each connection to the broker it turned the option on and read two timestamps for each message. The first, $t_{\mathrm{tx,so}}$, is the kernel's timestamp of the producer's write that carried the message out. The kernel returns it on the socket's *error queue*, where it leaves transmit timestamps for the program to collect, with the byte offset of the write's last byte. The second, $t_{\mathrm{rx,so}}$, is the kernel's timestamp of the packet that brought the message in, returned with the consumer's read. A driver takes the *software transmit timestamp* as it passes a packet to the network card. On these machines none came back, and the reason lies in the driver. In Linux 6.8, `hv_netvsc`, Hyper-V's network driver, hands each packet to the *accelerated-networking device*, a function of the physical card that Azure gives the machine directly (`netvsc_vf_xmit`, `drivers/net/hyperv/netvsc_drv.c` line 495). It then returns without reaching its own call to `skb_tx_timestamp()` (line 662). The drivers of that device take no such timestamp either. The transmit path of mlx5, the Mellanox driver, has no `skb_tx_timestamp()` before Linux 6.16, and that of MANA, the Microsoft Azure Network Adapter's driver, has none in Linux 6.8. So, as the plan allowed, the transmit timestamp was the one the kernel documentation names `SOF_TIMESTAMPING_TX_SCHED`, taken before the *packet scheduler*, the queueing discipline in front of the driver [S9]. The two socket timestamps give the kernel-timed latency $W_{\mathrm{so}} = t_{\mathrm{rx,so}} - t_{\mathrm{tx,so}}$, to set beside the captures' W. Any time a packet waits in the packet scheduler's queue therefore falls inside W_{so}.

All five predictions held (Table S15). The first asked that the sockets and the captures agree. In every setup the median of $\lvert W_{\mathrm{so}} - W \rvert$ was 2.4–3.0 μs, against the 20 μs allowed, and its 99th percentile at most 5.9 μs, against 100 μs. The receive timestamps were identical for 98.4% or more of each run's messages, and within 0.06 μs for 99% of them, so the difference is the transmit timestamp's. Taken before the packet scheduler, it came 2.3–3.2 μs before the capture's in each run's median, the time before the capture copies the packet. The second and third asked that W_{so} hold still as W did. Its median moved by at most 7.8% across load and priority, against 10%, and its 99th percentile by at most 12.2%, against 25%. The fourth asked that nothing be lost. None of the 239,424 messages had W_{so} at or below zero, and every message had both socket timestamps. The fifth asked that load still lengthen D in the same runs. At normal priority it raised the 99th percentile of D 4.2–4.7-fold, where the plan required 1.5-fold, and moved the mean of W_{so} by at most 5.1% of the rise in the mean of D, against 20%.

In three Kafka runs a pause held messages more than 150 ms, in D and in W_{so} alike, so the pause lay between the kernel's timestamps, not in the clients' threads. A setup's median sets those runs aside. The module copies every byte the clients write and read, in Python. That work falls in the clients' own time, not in W_{so}, which runs between two of the kernel's timestamps. Against freeze 39's runs, which shared the design but did not take the timestamps, the clients' own time, $D - W$, moved by -51 to $+76\,\mu$s on Kafka and $+15$ to $+45\,\mu$s on Redis, and the 99th percentile of D by -197 to $+183\,\mu$s, with both signs

and with the drift between two days included (`docs/results/so_fix/module_cost.csv`). The run-level intervals on the largest moves of W_{so} reach 14% at the median, and at the 99th percentile they have no useful upper end on Kafka, where the paused runs enter a setup's resampling. The plan, the register of every run and the reading are in the repository under `docs/results/so_fix`, read by `scripts/so_fix.py`. The module is `scripts/so_timestamps.py`.

Freeze 41 then gave the module a positive control and a rate ladder (S3.11).

For a tool's author, the module's choices are these. It asks for the transmit timestamp taken before the packet scheduler (`SOF_TIMESTAMPING_TX_SCHED`, or `TX_SOFTWARE` where the driver gives one), the software receive timestamp (`RX_SOFTWARE` with `SOFTWARE`), an identifier for each write (`OPT_ID`), and the timestamp alone on the error queue, without a copy of the packet (`OPT_TSONLY`) [S9]. It turns the option on at a connection's first write or read once the connection is up, so that the identifier counts bytes from the first byte written. On these machines the transmit timestamp came back twice for every write, and the module keeps the first. The kernel documentation says a packet yields one at each packet scheduler it passes, and `hv_netvsc` passes each packet through a second, the accelerated-networking device's own (`netvsc_drv.c` line 459). The module reads the error queue only when `poll()` reports something waiting there. On a socket with a timeout, as `kafka-python` sets one while it writes, Python waits for the socket to become readable before any read, so the module held each request past the client's own timeout and the pilot's first three Kafka attempts failed. Linux overwrites a read's receive timestamp with each packet the read consumes (`tcp_update_recv_tstamps`, called from the read loop in `net/ipv4/tcp.c`), so a read returns the timestamp of its last packet.

Three more points matter to a tool's author. First, a driver may report timestamps that it does not give. `hv_netvsc` reports software timestamps to `ethtool` (`ethtool_op_get_ts_info`, `netvsc_drv.c` line 2012), yet no software transmit timestamp came back, so a tool should check which timestamps actually arrive rather than trust `ethtool -T`. Second, on a stream socket a tool should set `SOF_TIMESTAMPING_OPT_ID_TCP` beside `OPT_ID`, as the kernel documentation advises, because without it the identifier's starting point is "not entirely trivial": it can be off by the bytes still queued when the option is turned on [S9]. The module does not set this flag. It needs no correction, because it turns timestamping on before the connection's first write. Third, writes that the kernel merges into one packet share one transmit timestamp, and the byte offsets show where that happened.

S3.11. Timed by the kernel, the acknowledgment reaches the client first on Kafka, and the delivery often does on Redis

Freeze 41 (plan version 44) ran what the earlier freezes had left open: the acknowledgment's own arrival, timed by the kernel; a positive control and a rate ladder for the kernel-timed latency; and two checks of the timescale. Its predictions were written and fingerprinted before any of its runs. A pilot then changed three instruments and the plan, and a dry run of the judge on the pilot's runs corrected two departures from the design, both before any judged run was read (`docs/results/order_scope/README.md`, `DESIGN.md`).

The clients kept a journal of every read and write on their broker connections, with the kernel's receive timestamp of each read, so each acknowledgment and each consumer request could be matched to its message by the broker's own wire format: Kafka's correlation identifiers, and Redis's replies in request order. From them come the acknowledgment's kernel-timed arrival $t_{\mathrm{rx,ack}}$, the *kernel-timed order* $S_k = t_{\mathrm{rx}} - t_{\mathrm{rx,ack}}$, which no client thread's delay enters, and the acknowledging thread's delay $\delta_{\mathrm{ack}} = t_{\mathrm{ack}} - t_{\mathrm{rx,ack}}$, so that $S = S_k + \delta_{\mathrm{recv}} - \delta_{\mathrm{ack}}$ for every message. A message carries the *pull flag* when the request it was delivered in answer to left after the message did. Block K ran 60 runs on the first x86 pair (both brokers, idle or at 88% load, at normal or real-time priority, and Redis with one publish worker instead of sixteen), and block R 24 there, with run-to-parity on or off at 88% load. On the second x86 pair, block P ran 24 with a delay of 0, 1 or 2 ms on the broker's path to the consumer, block L 32 at 50, 500 and $2{,}000$ messages a second, loaded at 50 only because the clients left no room for the load above it, and block I 16 at 88% load on the stock boot and with `idle=poll`. Every run passed the integrity rule at its first attempt but one, run again as the plan provides. Table S16 gives each prediction's verdict.

The order of the two arrivals. On Kafka the acknowledgment arrived first, by 72–92 μs at the setups' medians. S_k fell below zero for under 1% of the messages in 21 of its 24 runs and for at most 32% in the others, and under load 99.8% of its values of S below zero had $S_k \geq 0$, so they are late readings (K-a). In 16 of Redis's 24 runs with sixteen publish workers the delivery came first for most messages, by 16–51 μs at the medians of the setups where it did. Of Redis's values below zero, 95–100% had it first, at either priority and either load, and the design's rule counts those as the broker's reply order. With the delivery first, a value below zero needs no wait, only that the acknowledging thread's delay not fall short of the consumer's by more than the delivery's lead. A few Redis runs had most values of S below zero for the whole run, as did two of the eight ladder runs at 500 messages a second, one on each broker.

The delay itself. The acknowledging thread's 99th-percentile delay rose 17- and 30-fold under load and fell back at real-time priority (K-c). Read after the verdicts, Kafka's median delay, 163–180 μs in every setup, is the client's own processing. Its 99th percentile was 229 μs idle, 3.9 ms at 88% load and 274 μs at real-time priority. Under load at normal priority, the readings below zero had a median delay of $2{,}339$ μs on Kafka and 958 μs on Redis, against 171 and 58 μs for the rest, so they were read late. Message by message, the share of values below zero in each fifth of the margin lay within $1.14\times$ on Kafka and $1.85\times$ on Redis of the share the setup's own delays predict, $\Pr[\delta_{\mathrm{ack}} - \delta_{\mathrm{recv}} > S_k]$, but did not fall from one fifth to the next,

TABLE S16
FREEZE 41'S PREDICTIONS AND WHAT ITS RUNS GAVE. EACH SETUP'S VALUE IS THE MEDIAN OVER ITS RUNS UNLESS A PREDICTION POOLS MESSAGES (K-A, K-B); RATIOS ARE LOADED OVER IDLE, REAL-TIME OVER LOADED, OR THE RULE OFF OVER THE RULE ON. GENERATED AT BUILD TIME FROM THE COMMITTED RUN TABLES THROUGH THE BLOCK'S OWN JUDGE.

	Required	The runs gave	Verdict
K-v	median gap between the journal's and the capture's acknowledgment arrival at most $20\,\mu$s in every setup	largest median $0.0\,\mu$s	confirmed
K-a	Kafka: at least 90% of $S < 0$ with $S_k \geq 0$	99.8% of 4,024; 91.7% of 60	confirmed
K-b	Redis, real-time: at least half of $S < 0$ with $S_k < 0$	100.0% of 5,160; 99.8% of 4,098	confirmed
K-c	99th percentile of δ_{ack}: at least $1.5\times$ under load, at most half at real-time priority	Kafka $16.98\times$, $0.07\times$; Redis $30.01\times$, $0.03\times$	confirmed
K-d	at 88%, normal priority: the rate falls across the margin's fifths and lies within $2\times$ of the delays' prediction	Kafka within $1.14\times$, does not fall; Redis within $1.85\times$, does not fall	not confirmed
K-e	one publish worker against sixteen, Redis real-time	idle 0.04% against 1.44%; 88% load 0.06% against 0.04%	decision
R-a	rate of $S < 0$ without run-to-parity at most $2/3$ of the rate with it	Kafka $1.02\times$; Redis $0.91\times$	not confirmed
R-b	99th percentile of δ_{ack} without run-to-parity at most $2/3$	Kafka $0.99\times$; Redis $1.00\times$	not confirmed
P-a	W_{so} rises 0.9–1.1 ms per ms added	Kafka 0.93; Redis 0.95	confirmed
P-b	A moves at most 10% between delays	Kafka 18%; Redis 22%	not confirmed
P-c	D rises 0.9–1.1 ms per ms added	Kafka 0.94; Redis 0.95	confirmed
L-a	median $\lvert W_{\mathrm{so}} - W \rvert$ at most $20\,\mu$s at every rung	largest median $3.0\,\mu$s	confirmed
L-b	W_{so} within 10% under 88% load	50/s Kafka -6.3%; 50/s Redis $+25.7$%	not confirmed
L-c	at 50/s, at most 1% of messages flagged	at most 0.00%	confirmed
I	`idle=poll` halves Redis's share of S in $[-250, -150)\,\mu$s	4.2% to 5.7%	decision

because the margins are short beside the waits and the prediction is nearly flat across them (K-d). The paper's $p\,G(S_k)$, with p the share of acknowledgments read more than 0.1 ms after they arrived and G the rest of a slice of Fig. 3, put the share of values below zero at 99.6–99.9% on Kafka and 6.6–11.4% on Redis, against measured shares of 11.7–15.0% and 3.7–7.4%. On Kafka the client's processing alone outlasts 0.1 ms, so p counts it as a wait. Turning off run-to-parity moved neither the rate nor the delay (R). With one publish worker, Redis's real-time rate fell from 1.4% to 0.04% with no load, so there the interpreter lock the workers share sets most of it, but not under load (K-e).

The kernel-timed latency. A delay added on the broker's path to the consumer raised W_{so} 0.93–0.95 ms and D 0.94–0.95 ms for each millisecond added, while the publish latency, whose path the delay does not touch, moved by 18–22% between setups without following the delay, its runs scattering more than the 10% band (P). On the idle host the runs' median negative-span rate was 0.56, 0.38 and 3.7% on Kafka at 50, 500 and 2,000 messages a second, and 0.040, 18 and 0.74% on Redis. Kafka's reads drained more than one packet in 97, 81 and 75% of reads, Redis's in at most 0.01% (described, not judged), yet the sockets agreed with the captures at every rung (L-a). At 50 messages a second no message carried the pull flag, and at 500 at most 0.2% did. Under load at 50, Kafka's W_{so} held while Redis's rose 26%, in the captures as in the sockets, 20% of the rise in the mean of D, for a cause we have not traced (L-b). At 2,000 a second 96.2–99.7% of messages carried the flag, and their kernel-timed latency was 403–446 μs longer than the others'. Keeping idle processors spinning did not shrink Redis's mode of values near $-200\,\mu$s on that pair (4.2% of its values below zero on the stock boot, 5.7% with `idle=poll`), so there it is not the wake of an idle processor (I). In the paper's Fig. 2, from the Oracle runs, the same mode holds a far larger share of Redis's values below zero, which this test does not explain.

S4. THE POSITIVITY FILTER AND THE RETENTION LAW IN FULL

The benchmark's printed latency describes only the samples its publish schedule lets through the positivity filter, and each link in that claim can be checked against a source line, a counter or a registered prediction. Every benchmark run below is in embedded mode, with the whole benchmark in one JVM.

S4.1. Two source lines make the filter, and counting what they drop changes no printed number

What the positivity filter drops can be counted without changing the filter. In `LocalWorker.java`:294 the benchmark computes its end-to-end latency as

```
MILLISECONDS.toMicros(now - publishTimestamp)
```

from two millisecond timestamps, so the value, though in microseconds, is a whole number of milliseconds. In `WorkerStats.java`:95 the positivity filter,

```
if (endToEndLatencyMicros > 0)
```

decides whether that value reaches the histogram. Each message is counted as received; what the filter drops is not.

We add counters in an `else` branch of that condition and change nothing else: not the latency computation, not the filter, not any reported statistic. The benchmark's output is therefore exactly what it would be unpatched, and its printed summary can be set beside the retention behind it (Fig. 4b of the paper). The four counters, for zero, negative, most negative and kept samples, separate sign. A value below zero and an end-to-end latency shorter than a millisecond both fail a `> 0` test, and one unsigned total cannot tell them apart (S4.4). The patch is `external/omb/workerstats_discard_counter.patch`.

The counts are exact. The counters are shared across the JVM and their totals printed once at shutdown. A run that fails discards nothing, and its zero count would look measured, so a validity gate writes it as invalid, with its reason and no count. Unlike the benchmark's histograms, our counters include the one-minute warmup, yet a matched sweep with the warmup disabled leaves retention unchanged. The campaign totals 223 instrumented runs, swept one axis at a time with background load, message size, producer rate and warmup otherwise fixed.

S4.2. The phase of the publish schedule sets retention, as a 1970 counter note already showed

Where each publish instant falls within a step of the clock decides whether its sample survives. An end-to-end latency shorter than the step τ survives only if it crosses a grid instant, where the timestamp changes value. Otherwise both timestamps round down, or floor, to the same value, and the positivity filter deletes the zero difference (Fig. 4a of the paper). When phases are uniform, retention is $\theta = T_{\mathrm{true}}/\tau$, which is Equation 5 of the paper.

A paced producer does not draw its phases uniformly. If its publish interval T_{pub} satisfies $T_{\mathrm{pub}}/\tau = a/q$ in lowest terms, it visits only q distinct phases, so its retention is a whole count out of q. Call the retentions $0, 1/q, \ldots, 1$ the vertices of the configuration's q-lattice, and the interval between neighboring vertices, $100/q$ percentage points wide, a lattice step. At an incommensurate rate, whose ratio has no small denominator, the vertices are dense and retention is θ. Elsewhere, replicates of one configuration land on the vertices that bracket θ, and their spread obeys

$$\text{spread} \longrightarrow \begin{cases} 100/q & \text{when } \theta \text{ sits mid-step,} \\ 0 & \text{when } \theta \text{ sits on a vertex.} \end{cases} \tag{S1}$$

The step width is thus an upper bound on the spread, reached only when replicates realize both bracketing vertices. Retention at the incommensurate rates puts T_{true} at 0.49 ms, half a millisecond to within replicate noise (Section 5.3). One half is a vertex of every q-lattice with even q and of none with odd q, so the spread rule predicts full step width for odd denominators and almost none for even ones.

Hewlett-Packard's Application Note 162-1 [S12] carries this arithmetic for a counter timing an interval. For an interval of $Q + F$ clock periods it gives $\Pr[R = Q{+}1] = F$, which is Equation 5, and the denominator of the ratio of repetition rate to clock rate in lowest terms, our q, fixes how many distinct readings the counter can produce, so averaging N readings improves resolution by at most a factor $1/q$, not $1/\sqrt{N}$. Its cure, phase jitter on the repetition rate, becomes for a benchmark randomized publish instants (the paper's Table 4).

S4.3. Retention keeps to its lattice, and a registered change of message size moved one configuration onto a vertex and off it

The phase account predicts each configuration's behavior from its denominator alone. Its starkest case is $q = 1$: where the publish interval is a whole number of milliseconds, every publish has the same phase, and replicates of one configuration spread over almost the whole range, while those at incommensurate rates agree to within a few points (Table S17). The spread rule of Equation S1 holds in 10 of the 12 commensurate configurations. A spread above half the step width counts as full, so the test needs no tolerance. The two misses, 700 and 1250 msg/s, are predicted full and measured flat. Spread is a range over replicates that can each land on only two vertices, so a configuration shows its full width only if its replicates land on both. At $q = 5$ and $q = 7$ the upper vertex is the less likely of the two, so five replicates can all fall on the lower one even when the lattice holds exactly, and spread cannot tell that outcome from a refutation.

For that reason we built a second test after the campaign had run. It was not registered, and its first results were committed in the same change as its code. For each configuration, $\bar{d}$ is the mean distance of its replicates to the nearest vertex of its q-lattice, divided by the step width. The null is a continuum, in which replicates are drawn from a Normal distribution centered on the retention the incommensurate rates imply at the configuration's rate, with a measured standard deviation. Neither parameter is fitted to the configuration under test. The one-sided p-value comes from $4{,}000$ Monte Carlo draws at the observed number of replicates, and is Holm-corrected across all 12 configurations. Of these, ten are powered, meaning the null and the lattice predict different distances, and nine of those reject the continuum at $\alpha = 0.05$ after correction (Table S18), among them both configurations the spread rule missed, 700 msg/s at 0.009 and 1250 msg/s at 0.003. The tenth, 900 msg/s, rejects before correction ($p = 0.044$) and not after ($p = 0.131$), so we report it as unresolved. The two unpowered configurations, 400 and 800 msg/s, have lattices that pass through θ, where the two hypotheses coincide.

The registered test that manipulates the physics is the payload flip. Payload moves T_{true}, and so moves θ within its lattice step, with nothing else changed. Measured at an incommensurate rate (Table S19), 32 KB puts θ almost on the $2/3$ vertex of the $q = 3$ lattice and 64 KB puts it mid-step. The prediction was that this one manipulation would move a 300 msg/s configuration onto a vertex and off it again.

The spread flipped from full to flat and back, as predicted (Fig. S4). At 32 KB the five replicates spanned 62.06–75.69%, a spread of 13.6 points, and at 64 KB they spanned 73.06–99.78%, a spread of 26.7 points. Both cross the flat–full boundary at half the step width, 16.7 points, in the predicted direction. One registered detail failed: the 32 KB configuration pins at 69.2%, not on the vertex at 66.67%. The flip was one of six predictions registered with falsifiers before its campaign. It was confirmed, three were refuted by their falsifiers, one could not be evaluated, and one split (postmortem, S1.7).

TABLE S17

REPLICATE SPREAD FOLLOWS THE PHASE DENOMINATOR q IN 10 OF THE 12 COMMENSURATE CONFIGURATIONS. PAYLOAD, LOAD, DURATION AND HOST ARE FIXED. THE POSITION IS THE DISTANCE FROM THE MEASURED CONTINUOUS VALUE TO THE NEAREST VERTEX, AGAINST HALF THE STEP WIDTH. A CONFIGURATION IS PREDICTED FULL WHERE ITS POSITION EXCEEDS A HALF, AND SHOWS FULL WHERE ITS SPREAD EXCEEDS HALF THE STEP WIDTH. THE DAGGERED ROWS ARE THE TWO MISSES, RESOLVED IN TABLE S18. INCOMMENSURATE CONFIGURATIONS HAVE NO LATTICE STEP. THEIR SPREADS OF ABOUT THREE POINTS ARE THE NOISE FLOOR.

rate	T_{pub}/τ	q	n	width	pos.	predicts	spread	shows
1000/s	1/1	1	5	100.0	0.98	full	99.4	full
500/s	2/1	1	5	100.0	0.98	full	99.5	full
250/s	4/1	1	5	100.0	0.98	full	98.7	full
400/s	5/2	2	5	50.0	0.04	flat	17.6	flat
600/s	5/3	3	6	33.3	0.95	full	32.7	full
300/s	10/3	3	10	33.3	0.95	full	31.2	full
800/s	5/4	4	5	25.0	0.07	flat	7.2	flat
1250/s†	4/5	5	5	20.0	0.91	full	7.6	**flat**
625/s	8/5	5	10	20.0	0.91	full	19.3	full
875/s	8/7	7	5	14.3	0.87	full	10.7	full
700/s†	10/7	7	5	14.3	0.87	full	6.5	**flat**
900/s	10/9	9	5	11.1	0.84	full	6.9	full
incommensurate — phase set effectively continuous								
333/s	1000/333	>64	5	—	—	≈ 0	4.5	—
383/s	1000/383	>64	5	—	—	≈ 0	2.8	—
457/s	1000/457	>64	5	—	—	≈ 0	2.1	—
611/s	1000/611	>64	5	—	—	≈ 0	3.1	—
889/s	1000/889	>64	5	—	—	≈ 0	1.7	—
1053/s	1000/1053	>64	4	—	—	≈ 0	3.0	—
1219/s	1000/1219	>64	4	—	—	≈ 0	4.5	—

TABLE S18

RETENTION SITS ON THE q-LATTICE IN NINE OF THE TEN POWERED CONFIGURATIONS, TESTED AGAINST A CONTINUUM NULL. HERE $\bar{d}$ IS THE MEAN NORMALIZED DISTANCE TO THE NEAREST VERTEX, AND THE NULL COLUMN IS ITS MEAN UNDER THE SIMULATED NULL. THE COLUMN p_{Holm} IS A MONTE CARLO p-VALUE (4,000 DRAWS), HOLM-CORRECTED WITHIN ALL 12 CONFIGURATIONS; IT DECIDES THE VERDICT. FLAT (COINCIDENT) MARKS CONFIGURATIONS WHERE THE TWO HYPOTHESES PREDICT ALIKE AND THE TEST HAS NO POWER.

rate	a/q	n	$\bar{d}$	null	p_{Holm}	verdict
1250/s	4/5	5	0.131	0.351	0.003	grid
1000/s	1/1	5	0.003	0.479	0.003	grid
900/s	10/9	5	0.265	0.340	0.131	not resolved
875/s	8/7	5	0.209	0.382	0.003	grid
800/s	5/4	5	0.104	0.058	1.000	flat (coincident)
700/s	10/7	5	0.329	0.416	0.009	grid
625/s	8/5	10	0.112	0.447	0.003	grid
600/s	5/3	6	0.034	0.469	0.003	grid
500/s	2/1	5	0.043	0.490	0.003	grid
400/s	5/2	5	0.093	0.020	1.000	flat (coincident)
300/s	10/3	10	0.221	0.468	0.003	grid
250/s	4/1	5	0.181	0.489	0.003	grid

S4.4. Two floored clocks disagree by exactly one millisecond, so the sign of a discard names which kind of timestamp failed

The sign of a discarded sample says which clock failed, and the benchmark's two drivers show both kinds. The Kafka driver's 214 runs discarded 10,913,263 samples, and not one was negative. The Redis-driver replication's 9 runs carry all 41,403 of the campaign's negatives, every one exactly $-1000\,\mu$s.

The drivers explain the difference. The Redis driver takes its publish timestamp from `entry.getID().getTime()` in `RedisBenchmarkConsumer.java`:72, the stream-entry identifier the Redis server assigns on `XADD`. The Kafka driver uses a producer-set timestamp. So the Redis span subtracts a server-side timestamp from a consumer-side one, across two hosts, both floored to the millisecond, while the Kafka span floors two timestamps whose difference is taken inside one JVM.

Two clocks floored to the same resolution and offset by less than one millisecond can disagree by one millisecond and by no other amount. The only negative a floored cross-host difference can produce is therefore $-1000\,\mu$s, and every one of the 41,403 negatives is that value. Clock synchronization was clean at every logged minute, so this failure needs no misbehaving clock, only two clocks whose readings are floored. A difference taken inside one process has no second clock to disagree with, which is why the Kafka discards hold no negative. The same positivity filter thus hides a loss of resolution under Kafka and a clock failure under Redis, and an unsigned discard counter could not tell them apart.

TABLE S19
AT A RATE INCOMMENSURATE WITH THE MILLISECOND, RETENTION READS OFF THE END-TO-END LATENCY. RETENTION AGAINST PAYLOAD AT 457 MSG/S, WHERE SAMPLES STAY DEPHASED. THE IMPLIED T_{true} INVERTS EQUATION 5. UP TO 8 KB THE ADDED DELAY IS COMPARABLE TO THE REPLICATE NOISE; THE 32 AND 64 KB LEVELS CARRY THE RESULT.

Payload	Retention (3 reps)	Median	Implied T_{true}
200 B	51.08, 52.03, 52.88	52.03%	0.520 ms
1 KB	50.11, 50.32, 51.47	50.32%	0.503 ms
2 KB	52.13, 54.62, 54.77	54.62%	0.546 ms
4 KB	51.86, 54.17, 56.24	54.17%	0.542 ms
8 KB	54.92, 55.22, 57.89	55.22%	0.552 ms
32 KB	68.01, 68.44, 69.40	68.44%	0.684 ms
64 KB	84.21, 85.36, 86.82	85.36%	0.854 ms

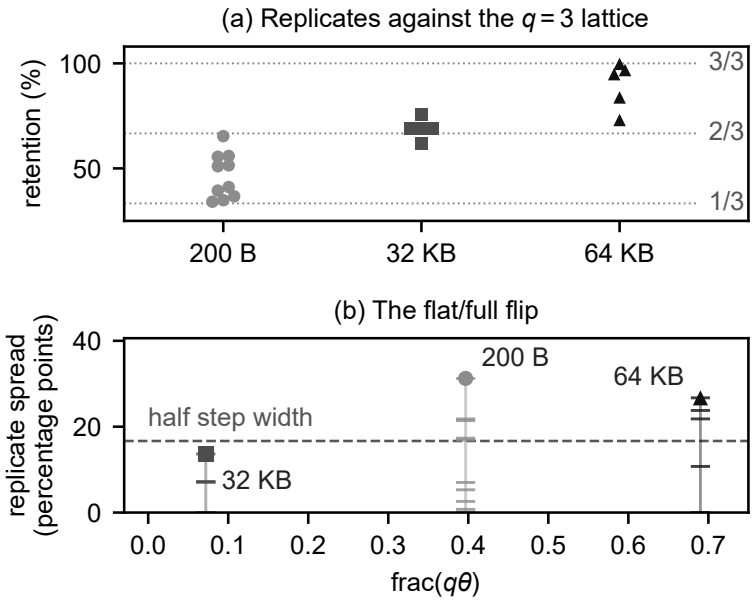


Fig. S4. **One manipulated variable moves a configuration onto its lattice and off it.** (a) Replicates at 300 msg/s ($q = 3$) by payload, against the thirds lattice: 200 B straddles both branches, 32 KB pins near the $2/3$ vertex, and 64 KB re-opens to the upper step. (b) Replicate spread against $\mathrm{frac}(q\theta)$, each stick showing the replicates its spread is made of, with θ each configuration's own mean retention in this campaign. The registered prediction placed 32 and 64 KB at 0.055 and 0.56 from the payload sweep's θ. The flat–full boundary at half the step width is crossed in both directions.

S4.5. An independent harness reproduces the deletion, and the benchmark's distributed mode, which failed on the cloud testbed, ran in the registered campaign

The deletion does not depend on the benchmark's code. We reproduced the positivity filter in a Python harness that shares no code with the benchmark, with the same timestamp, subtraction and filter. On one clock it recorded 0 negative differences across 905,040 samples in 12 runs, as a causal chain requires, and the grid reappeared intact. Across two hosts with disciplined clocks, 12 runs again recorded 0 negatives in 905,040 samples and an intact grid, but the deleted fraction followed the drift of the clock offset. Of the three rates, only the incommensurate one, 457 msg/s, has no lattice spread wide enough to hide the drift. Over four replicates there, cross-host retention wandered from 13.4 to 26.9%, while on one clock the same configuration's replicates spanned 0.98 points.

On the cloud testbed the benchmark's own distributed mode, where its subtraction spans two clocks, could not be measured. It never completed a run in eleven attempts: eight at three background-load levels, including none, and three more run with full diagnostics. The unloaded attempts failed like the loaded ones, which refutes our own hypothesis that processor starvation was the cause, and the three diagnosed attempts failed identically, in the coordinator-to-worker path before any workload started. The first attempt, made before the validity gate of S4.1 existed, wrote a vacuous zero that the artifact keeps marked invalid, and the gate refused every later attempt with its reason. On that testbed the distributed mode's exposure to the deletion therefore rested on reading its source. The timestamp is written on the producer's host and read on the consumer's, and the positivity filter drops every difference at or below zero.

On the registered campaign's machines the distributed mode ran unmodified in October 2026, in the campaign's tools block (S5). A coordinator and two workers, one producing and one consuming, ran on one machine against a Kafka broker on the other. The benchmark's median followed a delay added to the path. We moved the consuming worker's clock back at two

offsets of a few milliseconds, and the filter dropped whatever then computed to zero or below. In three of the four runs at the smaller offset the median fell to 1 ms, the smallest value a surviving sample can take. Elsewhere only the slow tail survived, and the median rose to the tail's; in one run nothing survived to print. The unmodified benchmark counts nothing it drops, so these runs show its filter at work across two processes but do not measure the deletion, which rests on the embedded mode and on the harness above.

S5. Tools read at source, tools run, and public reports

How the tools were chosen. No rule fixed in advance chose the tools; they were chosen in rounds. The first round is the list in the experiment plan (`freezes/04-experiment-plan`, Section "The tools"): the tools that brokers' projects ship, and widely used load generators. It holds the OpenMessaging Benchmark, Kafka's bundled end-to-end and producer tools, Apache Pulsar's performance client, `emqtt-bench`, RabbitMQ PerfTest, the NATS command-line tool's latency test and `nats bench`, librdkafka's `rdkafka_performance` and the Valkey benchmark; the load generators `wrk2`, `memtier`, YCSB, `k6`, Vegeta and `hey`; and `fio`, `btt` and Rezolus, kept as readings of source because they time no message. We audited that list by the plan's protocol on 19 September 2026 (`data/tools_audit/batch_01.json` to `batch_09.json`; the protocol is in `data/tools_audit/README.md`). Two more rounds, read in October 2026 after the first round's tools had run (`batch_10.json` and `batch_11.json`, both dated 11 October), reached tools around Kafka, other brokers' benchmarks, and benchmarks of MQTT, cloud messaging and stream processing. The second round added Xinfra Monitor, Strimzi Canary, Kafka's Trogdor, RabbitMQ Stream PerfTest, the Yahoo Streaming Benchmark and PerfKit Benchmarker's messaging benchmark. The third added Redpanda's `kgo-repeater`, IBM Sarama's producer tool, Pepper-Box, SBK, `mqtt-benchmark`, Google's Pub/Sub load test, xk6-kafka, the Pravega benchmark, RocketMQ's benchmark, Apache Artemis's perf client, Quiver and Apache BookKeeper's benchmark. The set is therefore not a random sample of the field's tools, and its counts describe these tools, not the field. The *registry*, a file with one source line for each verdict, holds the 27 tools whose handling of a value at or below zero was classified from a line of their source. It leaves out Pepper-Box and xk6-kafka, which do not make the measurement (the last note below).

Tools read at source. Of the 27 tools we read at source, 24 time messages, and 18 of those message tools cut a value at or below zero out of a figure they report without counting it. `emqtt-bench` is among them because it leaves the value out of the average it prints, though its histogram keeps it. So is Kafka's bundled end-to-end tool, which times a message that reached the consumer first as arriving after its acknowledgment (the note on `EndToEndLatency` below). SBK, the Storage Benchmark Kit, cuts the value but counts it, the one message tool that does. Of the remaining five message tools, which cut nothing, IBM Sarama's producer tool and `mqtt-benchmark` time only to the acknowledgment, on a monotonic clock, and never meet it, and PerfKit Benchmarker's messaging benchmark, Redpanda's `kgo-repeater` and RocketMQ's benchmark keep it. Counted instead by whether a tool loses or replaces the value, wherever it times, 19 of all 27 tools read at source dispose of it without counting it: every message tool above that cuts it uncounted except `emqtt-bench`, whose test of the sign protects a counter rather than the sample, and two of the three tools that time no message, `fio` and `btt`. The paper's Table 3 leaves out `fio`, `btt` and Rezolus because they time no message, and the notes below describe all three. Every verdict traces to a line of source a reader can check. Each row of the registry file, `docs/results/external/harness_registry.csv`, carries one source line, the file and symbol it came from, the upstream URL and the date it was read. None of the tools' maintainers had been told of these findings when this was written, and the source line beside each finding is there so that a maintainer can check it. The class labels are not stored. They are recomputed on every build and every test run by the classifier that read the OpenMessaging Benchmark a month before the other tools were audited (`classify()` in `scripts/audit_external_harness.py`), so a reader who disagrees with a verdict can check the line against the pattern that matched it. The classifier's patterns were written against JVM sources and had to be extended before they could read an Erlang guard or clock call, librdkafka's guard in C, or the spellings of the tools read in October. They see only what they recognize, so the registry's count of disposals is a lower bound on the disposals in these tools.

The count also rests on choices of ours, and a reader may reject some of them. Table S20, generated from `docs/results/external/audit_sensitivity.csv`, counts again with one kind of cut taken out at a time, each row keeping the exclusions of the rows above it. The count falls from 19 of the 24 message tools to twelve, and in every reading all but one of the tools that cut count none of what they cut.

We call a tool's own conditional on the sign of an interval its *guard*. A guard is a deletion only if what it guards is the sample itself, not a counter. Table S21 is the registry, one row per span because one tool can treat two spans differently, generated from the registry file at build time. Its sources, in row order, are [S13]–[S15], [S15]–[S20], [S20]–[S22], [S22]–[S37], [S37]–[S39].

The 19 silent disposals take three forms. A filter drops what its comparison excludes and reports no count of what it removed. The test > 0 deletes every sample that computes to exactly zero, while ≥ 0 keeps those and drops only values below zero. The benchmark and librdkafka's `rdkafka_performance` filter at > 0. Apache Pulsar's performance client guards at ≥ 0, as do RabbitMQ Stream PerfTest and the readers two other tools ship, so the zeros the benchmark deletes pile up visibly at zero. A substitution keeps the sample count, so any retention computed downstream reads 100%, which makes it harder to detect than a filter. A library refusal leaves the sample unrecorded because the tool never checks whether its library rejected it.

TABLE S20

UNDER THE STRICTEST READING, TWELVE OF THE 24 MESSAGE TOOLS STILL CUT A VALUE AT OR BELOW ZERO. CUT: THE MESSAGE TOOLS THAT REMOVE SUCH A VALUE FROM A FIGURE THEY REPORT. OF THEM, UNCOUNTED: THOSE THAT COUNT NONE OF WHAT THEY REMOVE. EACH ROW TAKES OUT ONE MORE KIND OF CUT, KEEPING THE EXCLUSIONS OF THE ROWS ABOVE IT.

Reading of the classification	Cut, of 24	Of them, uncounted
As counted	19	18
Less a zero dropped where no negative arises	18	17
Less a replacement made by the order of two calls	17	16
Less a disposal by a script or notebook the project ships	15	14
Less a disposal the tool announces or crashes on	12	11

TABLE S21

OF THE 27 TOOLS WE READ AT SOURCE, 19 DISPOSE OF A VALUE AT OR BELOW ZERO WITHOUT COUNTING IT. THE SPAN EACH MEASURES, THE CLOCK IT READS, WHAT HAPPENS TO A VALUE AT OR BELOW ZERO, AND WHETHER THE TOOL COUNTS WHAT IT DISCARDS; ONE ROW PER SPAN WHERE A TOOL MEASURES TWO, AND A DASH WHERE NOTHING IS DISCARDED. MAINTAINER: THE ORGANIZATION OR PERSON THAT MAINTAINS THE TOOL; 14 OF THE 20 MAINTAINERS ARE ORGANIZATIONS.

Tool	Language, maintainer	Span measured	Clock	What happens to the value	Counts
Artemis `perf`	Java, Apache Software Foundation	end-to-end	microsecond	refused by own library	no
BookKeeper benchmark	Java, Apache Software Foundation	publish	nanosecond	filter, > 0	no
Apache Kafka (bundled)	Java, Apache Software Foundation	end-to-end	nanosecond	truncate to the resolution; substitute a value	no
Apache Kafka (bundled)	Java, Apache Software Foundation	publish	millisecond	truncate to the resolution	no
Apache Pulsar perf	Java, Apache Software Foundation	end-to-end	millisecond	filter, ≥ 0	no
RocketMQ benchmark	Java, Apache Software Foundation	end-to-end	millisecond	—	—
Pub/Sub load test	Java, Google	end-to-end	millisecond	substitute a value	no
Sarama producer perf.	Go, IBM	publish	millisecond	truncate to the resolution	no
Kafka Trogdor	Java, Apache Software Foundation	end-to-end	millisecond	substitute a value	no
Kafka Trogdor	Java, Apache Software Foundation	publish	millisecond	refused by own library	no
NATS CLI	Go, Synadia	end-to-end	nanosecond	refused by own library	no
OpenMessaging Bench.	Java, Linux Foundation	end-to-end	millisecond	filter, > 0; truncate to the resolution	no
OpenMessaging Bench.	Java, Linux Foundation	publish	nanosecond	—	—
PerfKit messaging	Python, Google	end-to-end	millisecond	truncate to the resolution	no
Pravega benchmark	Java, Pravega	end-to-end	millisecond	refused by own library	no
Quiver	Python, Justin Ross	end-to-end	millisecond	substitute a value	no
RabbitMQ PerfTest	Java, Broadcom/VMware	end-to-end	nanosecond	substitute a value	no
RabbitMQ Stream PerfTest	Java, Broadcom/VMware	end-to-end	millisecond	filter, ≥ 0	no
`kgo-repeater`	Go, Redpanda Data	end-to-end	microsecond	—	—
Rezolus	Rust, IOP Systems	scheduler wakeup	nanosecond	discard	yes
SBK	Java, Keshava Munegowda	end-to-end	millisecond	discard	yes
Strimzi Canary	Go, Strimzi	end-to-end	millisecond	filter, ≥ 0	no
Xinfra Monitor	Java, LinkedIn	end-to-end	millisecond	substitute a value	no
Yahoo Streaming Bench.	Clojure, Yahoo	end-to-end	millisecond	filter, ≥ 0	no
`blktrace/btt`	C, Alan D. Brunelle	block stage	nanosecond	substitute a value	no
`emqtt-bench`	Erlang, EMQ Technologies	end-to-end	millisecond	kept; zero in its printed average	no
`fio`	C, Jens Axboe	I/O completion	nanosecond	substitute a value	no
`rdkafka_performance`	C, Confluent	end-to-end	microsecond	filter, > 0	no
`rdkafka_performance`	C, Confluent	publish	microsecond	—	—
`mqtt-benchmark`	Go, Alexander Krylovsky	publish	nanosecond	—	—
wrk2	C, G. Tene	end-to-end	microsecond	refused by own library	no

Truncation is a separate loss: it keeps every sample but loses precision. The benchmark truncates before it filters, and Kafka's bundled tools and PerfKit Benchmarker's messaging benchmark only truncate. The notes below explain the verdicts a reader may question, with the versions and dates of the changes we found; the postmortem, S25.2, keeps a note on every tool.

- The OpenMessaging Benchmark appears twice because it treats its two spans differently: it times its end-to-end latency on a millisecond clock and filters it, and times its publish latency in nanoseconds and records every sample (Section 5.3). librdkafka's `rdkafka_performance` appears twice too: its consumer keeps its own wall clock less the producer's timestamp only if the difference is above zero and under five minutes, printing a line for each value it drops but keeping no count, and its producer records every value.
- The `EndToEndLatency` tool bundled with Kafka cannot produce a value below zero, because it uses one thread, one monotonic clock and a synchronous publish call. It loses the measurement anyway, in two ways. Each sample prints in floating point, but the `long[]` its percentiles come from is filled by the integer division `elapsed / 1000 / 1000`, so on a path faster than a millisecond its p50, p99 and p999 all read zero. And it polls for a message only once its publish call has returned the acknowledgment, so a message that reached the consumer first is timed no earlier than its acknowledgment, a later time standing in for its arrival. Nothing counts it; the registry classes that as a substitution, and K1

below measured how often. The companion `ProducerPerformance` takes its publish-to-acknowledgment difference on `System.currentTimeMillis()` and casts it to `int`.

- Three verdicts rest on a zero that is not a negative, or on what a project ships beside its tool. BookKeeper's benchmark keeps a publish latency only if it is above zero, because a zero also marks an add that never completed; its clock is monotonic, so no negative can arise. Strimzi Canary bins a negative with its fastest messages, and the script it ships to read its latency log matches durations with `[0-9]+`, which cannot read a minus sign. The Yahoo Streaming Benchmark writes every value, but the notebook it ships keeps only windows whose last write, on the job's clock, comes at or after the window's end on the generator's clock, on another host. A user reported values such as $-70{,}646$ ms in 2018; the replies put them down to unsynchronized clocks across worker nodes, and the report is still open.
- Three tools changed their handling recently. `emqtt-bench`'s `> 0` guard protects a counter and leaves its histogram alone; the guard that once deleted from the histogram was removed in January 2025. Its printed average counts a value at or below zero as zero. RabbitMQ PerfTest's `getDifference` returns the absolute value of the difference, so a negative span enters its metrics as a positive one, uncounted. From version 2.0.0 of 2017 until version 2.25.0 of June 2026 its metrics recorded only values above zero, deleting every trip shorter than a millisecond in its millisecond mode. RabbitMQ Stream PerfTest has dropped values below zero since 2 June 2026, release 1.7.0, its fix for a user's report of a negative latency during a rolling restart; before that, the library it records into refused them and warned once.
- In `fio` three timing functions return zero for a negative interval, under the comment "time warp bug on some kernels?", and in `btt` `tdelta()` returns one nanosecond for stage timestamps out of order; neither records that it fired. Rezolus counts the samples its scheduler sampler discards, those whose two timestamps arrived out of order across CPUs, in a metric added on 12 August 2026; its other samplers have none.
- Four further tools we read are not in the count, because they do not make the measurement. SPECjms2007 defines Delivery Time and never reports it, Redpanda's `rpk` ships no latency benchmark, Pepper-Box times only the call handing a message to Kafka's client, and xk6-kafka, four of whose latency figures are always zero, times whole calls of its script.

Tools run. Reading a tool tells what it does with a value below zero; running it tells whether it ever meets one. The registered law campaign ran 32 benchmarking tools, four rounds each, in nine blocks, under predictions fixed from their source before any run. Table S22 gives each tool's row: what it times, what its reading at source found, its results in the tests below, the freeze and plan version that registered its block, the day the block began and the x86 pair it ran on. Each block's plan was public before its runs, the two pairs are of one machine type and processor, and every amendment that touched a block is listed in `freezes/README.md`. PerfKit Benchmarker's messaging benchmark timed an Azure Service Bus queue and the Pub/Sub load test Google's Pub/Sub, from a Google Cloud project; the queue and the project were made for their runs and deleted once the runs were home. The others ran against a broker on the second machine of their pair. The postmortem, S25.3, tells how each block was set up and gives every run's story in full.

The tests. T1 added a delay to the path in steps, a staircase. It cannot reach a managed service, so PerfKit and the Pub/Sub load test ran its zero step alone. Every other tool's figure but the Yahoo Streaming Benchmark's and RocketMQ's followed the delay at a slope inside the band the plan fixed in advance. T2 used libfaketime, which shifts the clock a process reads, to move back the clock of the process that subtracts, at two offsets longer than the interval the tool times. A program registered with the plan in freeze 21, the judge, then chose for each run among five behaviors: keeps every value, drops the negatives, drops zero and below, replaces the negatives with zero, and times against one clock. The second block exposed two faults in the judge. It ruled a tool that kept fewer values than it was asked to publish out of keeping every value but not out of timing against one clock, which keeps every value too. Thus a run of Pulsar's client that kept 7 of 3,000 latencies was judged to time against one clock. And it predicts each behavior's figures from the trips of a reference, our own client for the same server, so a tool with a slow tail of its own can be ruled out of the behavior it shows. Version 34 (`freezes/31-experiment-plan`), public before either block was judged again, corrects the first fault and decides how many clocks a tool reads from the offset alone, without the reference. A tool reads two if it lost values or its average, median or minimum moved by more than half the offset, one if it kept every value and none of those figures moved by a quarter of it, and is undecided otherwise. Every T2 run of both blocks was judged again under both rules, and the corrected rule changed two verdicts, both for runs of Pulsar's client: the run above became undecided, and an undecided one became "drops the negatives". A tool counts below as reading one clock, or two, only if the rule decided some of its runs and every run it decided agrees with the clocks its interval implies.

Of the tools run, 19 read both timestamps from one clock, and moving the clock left their figures within a quarter of the offset in 151 of their 152 runs; in the other, SBK's minimum, printed in whole milliseconds, moved by one. All but PerfKit take both timestamps in one process. PerfKit's publisher and receiver are two processes of one program on one machine, and T2's offset moved the clock of both. Freeze 21's judge, whose allowance for noise comes from a staircase PerfKit has none of, left all its clock-moved runs undecided, and version 34's rule answers one clock in every one, as version 37 predicted. The ten tools counted as reading two clocks timestamp a message in one process and subtract in another, `kgo-repeater` on another machine, and in 73 of their 80 runs the moved clock shifted their figures or cut what they kept. Pulsar's client drops a value below zero: under the larger offset it kept 3 to 7 of 3,000 latencies, yet its summary printed 3,000 records received beside figures computed from those few. `emqtt-bench`'s histogram kept every value, but its console printed no latency in any offset run, because it prints its once-a-second average only when a counter of the latencies above zero changes. Kafka's

TABLE S22
OF THE TOOLS RUN, VERSION 34'S RULE PLACED 19 ON ONE CLOCK AND TEN ON TWO, AND COULD NOT PLACE THREE. INTERVAL: TWO HOSTS AND TWO PROCESSES, A PUBLISH TIMED TO ITS RECEIPT IN ANOTHER PROCESS, ON ANOTHER MACHINE OR THE SAME ONE; ONE HOST, THE SAME ON ONE CLOCK; ONE PROCESS, A PUBLISH TIMED TO ITS RECEIPT WITHIN ONE PROCESS; TO ACK., A PUBLISH TIMED TO ITS ACKNOWLEDGMENT; ROUND TRIP, A REQUEST TIMED TO ITS REPLY. AT OR BELOW ZERO, AS READ AT SOURCE: FILTER, DROPPED WHERE THE COMPARISON FAILS; SUBSTITUTE, REPLACED BY ANOTHER VALUE; REFUSE, REFUSED BY THE TOOL'S OWN LIBRARY; TRUNCATE, ROUNDED DOWN TO WHOLE UNITS OF ITS CLOCK; ZERO IN AVERAGE, KEPT BUT AVERAGED AS ZERO; DISCARD, DROPPED AND COUNTED; —, KEPT. COUNTED: WHETHER IT COUNTS WHAT IT REMOVES. T1 SLOPE: THE FIGURE'S MOVE PER MILLISECOND OF DELAY, PER CROSSING, LOWEST AND HIGHEST ROUND (—: A MANAGED SERVICE). T2: THE NUMBER OF CLOCKS ITS INTERVAL IMPLIES, AND OF ITS RUNS WITH THE CLOCK MOVED BACK, HOW MANY VERSION 34'S RULE ANSWERED WITH THAT NUMBER. S1 HELD: ROUNDS IN WHICH THE STEP TEST'S PREDICTION HELD (STREAM PERFTEST AS ONE INSTANCE). FREEZE (PLAN), BEGAN, PAIR: THE BLOCK'S FREEZE AND PLAN VERSION, FIRST DAY AND X86 PAIR.

		At or		T1	T2		S1	Freeze		
Tool	Interval	below zero	Counted	slope	Clocks	Runs	held	(plan)	Began	Pair
kgo-repeater	two hosts	—	—	0.97–1.09	two	8/8	—	35 (38)	11 Oct	first
Artemis perf	two processes	refuse	no	0.96–1.06	two	0/8	4/4	36 (39)	11 Oct	second
emqtt-bench	two processes	zero in average	no	0.90–1.05	two	8/8	—	30 (33)	10 Oct	first
Kafka Trogdor	two processes	substitute	no	1.14–1.14	two	4/8	—	32 (35)	11 Oct	first
librdkafka perf.	two processes	filter > 0	no	0.96–1.05	two	8/8	—	21–22 (25–26)	25 Sep	first
OpenMessaging Benchmark	two processes	filter > 0; truncate	no	0.56–1.04	two	5/8	—	30 (33)	10 Oct	first
Pub/Sub load test	two processes	substitute	no	—	two	0/8	3/4	38 (41)	13 Oct	second
Pulsar perf.	two processes	filter ≥ 0	no	1.03–1.14	two	8/8	—	30 (33)	10 Oct	first
Quiver	two processes	substitute	no	0.90–1.48	two	8/8	4/4	37 (40)	12 Oct	second
RocketMQ benchmark	two processes	—	—	1.78–4375.24	two	8/8	—	36 (39)	11 Oct	second
Stream PerfTest	two processes	filter ≥ 0	no	1.03–1.14	two	8/8	4/4	32 (35)	11 Oct	first
Yahoo Streaming Bench.	two processes	filter ≥ 0	no	-0.24–235.52	two	8/8	—	33 (36)	11 Oct	second
PerfKit messaging	one host	truncate	no	—	one	8/8	4/4	34 (37)	11 Oct	first
Kafka end-to-end	one process	truncate; substitute	no	1.01–1.02	one	8/8	4/4	21–22 (25–26)	25 Sep	first
NATS latency	one process	refuse	no	0.99–1.06	one	8/8	—	21–22 (25–26)	25 Sep	first
Pravega benchmark	one process	refuse	no	1.03–1.14	one	1/8	4/4	37 (40)	12 Oct	second
RabbitMQ PerfTest	one process	substitute	no	0.97–1.00	one	8/8	—	21–22 (25–26)	25 Sep	first
SBK	one process	discard	yes	0.58–0.91	one	7/8	4/4	35 (38)	11 Oct	first
Strimzi Canary	one process	filter ≥ 0	no	1.00–1.00	one	8/8	4/4	32 (35)	11 Oct	first
Xinfra Monitor	one process	substitute	no	1.01–1.06	one	8/8	4/4	32 (35)	11 Oct	first
BookKeeper benchmark	to ack.	filter > 0	no	0.73–1.20	one	8/8	—	36 (39)	11 Oct	second
Kafka producer	to ack.	truncate	no	1.00–1.14	one	8/8	—	21–22 (25–26)	25 Sep	first
mqtt-benchmark	to ack.	—	—	1.00–1.01	one	8/8	4/4	35 (38)	11 Oct	first
Sarama producer	to ack.	truncate	no	1.01–1.34	one	8/8	0/4	35 (38)	11 Oct	first
hey	round trip	not read	—	1.00–1.04	one	8/8	—	21–22 (25–26)	25 Sep	first
k6	round trip	not read	—	1.00–1.02	one	8/8	—	21–22 (25–26)	25 Sep	first
memtier	round trip	not read	—	0.96–1.00	one	8/8	—	21–22 (25–26)	25 Sep	first
NATS bench	round trip	not read	—	1.00–1.03	one	8/8	—	30 (33)	10 Oct	first
Valkey benchmark	round trip	not read	—	0.98–1.05	one	8/8	—	21–22 (25–26)	25 Sep	first
Vegeta	round trip	not read	—	0.99–1.00	one	8/8	—	21–22 (25–26)	25 Sep	first
wrk2	round trip	refuse	no	0.92–1.04	one	8/8	—	21–22 (25–26)	25 Sep	first
YCSB	round trip	not read	—	1.00–1.04	one	8/8	—	30 (33)	10 Oct	first

Trogdor replaces a value below zero with zero, so a moved clock can lower its figures only by its own latency, less than half the larger offset, and its runs at that offset are undecided. The judge read librdkafka's `rdkafka_performance` as keeping every value in three of its four rounds, but no line reporting a dropped value appears in any of its runs and none of its values came near zero, so that verdict tested nothing. The rule could not place three tools, the Pravega benchmark, Apache Artemis's perf client and the Pub/Sub load test, which are in neither count.

Six tools whose results need explaining. The Pravega benchmark's count, not its clock, misled the rule. It takes both timestamps in one process, and its T2 moved the machine's own clock instead, because libfaketime slows this Java tool. Its median then stayed where its control's was in every run, and its average moved by at most 0.3 ms. But it counts only what arrives in a fixed minute, set-up included, so in all but one run its count fell 43 to 44 short of the $3{,}000$ asked for, as its controls' did, and version 34's rule reads a count shorter than its control's as values dropped. It answered two clocks in the five of the eight runs whose count came out one below its control's, undecided in the two whose counts matched, and one clock only in the one run whose count was whole.

Apache Artemis's perf client reads two clocks but hid the offset by having each refused value delivered again. Its histogram throws before it counts a value below zero, so the client logs AMQ132001 and rolls the message back, and the broker delivers it again, up to ten times, until it comes out at or above zero. Under the offsets it received $5{,}992$ to $8{,}972$ messages for the $3{,}000$ it published and logged $2{,}992$ to $5{,}972$ such warnings, yet reported success, as version 39 predicted. Its histogram kept the deliveries that came out above zero, many a little above, so its median moved by less than half the offset, and version 34's rule answered one or undecided in all eight runs, never two.

The Pub/Sub load test printed nothing under the offsets, because its controller threw first. It counts each value into a bucket by the logarithm of its milliseconds, so every value at or below a millisecond, a value below zero included, falls into the first

bucket, with the fastest. Under offsets of 26.6 to 89.5 ms (D41-3) that bucket held 87 to 120 of the 120 values its subscriber counted in each run, and its search for the median fails once the bucket holds half the values. In every run its controller threw an exception reading "Programming error: negative values appear in buckets.", printed no figure for its subscriber and wrote no results file, as version 41 predicted (T2-l), while each control printed its figures. Its controller also prints each percentile as the bounds of the bucket one below the bucket that holds it. Its T4 run, a test that moved its subscriber's clock a second ahead and so raised every value by 1,000 ms, printed in every round its median's range as 656.8 to 985.3 ms, wholly below a second (T4-l).

Quiver's unsigned subtraction wraps a value below zero to some 10^{19} ms. Its summary has subtracted the two timestamps of each message, which it calls a transfer, in unsigned 64-bit integers since December 2021, so a value below zero becomes 2^{64} less its size, about 1.8×10^{19} ms. Under the offsets its maximum wrapped in every run, and at the larger offset 2,995 to 2,999 of its 3,000 transfers fell below zero, so its median and average wrapped too. Version 40's prediction that its maximum and average would both pass 10^{18} ms at both offsets (T2-q) held in three rounds and failed in one. In that round libfaketime slowed its receiver by about a millisecond, so that its trip grew as long as the smaller offset and only 1 transfer fell below zero there. Neither judge has a behavior for a value that wraps, so both left its runs undecided; version 34's answer from the offset alone was two in every run.

The Yahoo Streaming Benchmark's figure ran far ahead of the delay, because each write of a window waits for replies that cross it. Its slopes ran from -0.2 to 236 ms per millisecond added, outside the band as version 36 predicted, but not for its reason, that a window's figure, the time of its last write to Redis less the window's end, is set by the once-a-second flush alone. The flush writes the hundred advertising campaigns' windows one after another, and each write waits for four replies from Redis, each crossing the delay. The writes of one window for successive campaigns were 3 ms apart with no delay added and 10 to 11 ms apart with 2 ms added, a gap growing by 3.1 to 3.7 ms per millisecond added (`scripts/ysb_post_hoc.py`). A millisecond added could thus move a late campaign's figure by hundreds of milliseconds. With no clock moved, its notebook drops 20–26% of windows, those whose last write came before the window's end (D36-6), so both judges, whose reference drops nothing, failed on it. Freeze 21's judge, whose allowance for a change is three times the figure's scatter across the staircase, which the flush made hundreds of milliseconds wide, read five of its eight clock-moved runs as one clock; version 34's left all undecided, though its answer from the offset alone is two. Read after the runs against its own reference, "drops the negatives" fits every run, though the verdicts stand as given. It predicts 49–50% of windows kept, against 46–58% kept, and averages within 38 ms of those printed. Every run's average fell by 179 ms or more from its control, where one clock predicts no fall (`docs/results/tools/ysb_t2_fit.csv`).

RocketMQ's benchmark followed the delay in no round, against version 39's prediction, because its consumer first waited for its queue and then crossed the delay twice (`scripts/rocketmq_post_hoc.py`). Its producer, which has no rate of its own, published its 3,000 messages in about three seconds, while its consumer is given the run's new queue only when its client next balances its queues, every 20 s. In six of its 76 runs the broker therefore held the messages until then: five staircase runs averaged 19.4 to 20.3 s, all of it after the broker had stored them, where no run that did not wait averaged more than 5.2 ms. The other steps rose by 1.78 to 2.04 ms per millisecond added, and the time from the broker's store of each message to the consumer, which leaves the producer out, by 1.63 to 1.97. Its consumer asks for messages again only when an answer comes, so a message stored just after an answer has left waits for the next request, which crosses the delay, and then crosses it itself: two crossings, where the prediction counted one.

Load. A third test, T3, ran each tool idle and against a background load that kept every core 88% busy, each with and without real-time priority on the process that subtracts. The plan fixed no rule for T3, so what follows describes the runs and judges nothing. The load lengthened the tail of every tool whose 99th percentile can show a wait for a core (Table S23). Taking the median over the four rounds, the 99th percentile of each of these 25 tools rose by 0.9–7.0 ms, of the order of the 2.8 ms slice a thread can wait for a core (Section 4.3), and real-time priority took back at least nine tenths of that rise in 17 of them. The median moved by at most 0.2 ms in all but four, Trogdor, SBK, Quiver and the Pravega benchmark, which print their medians in whole milliseconds. Each of their medians rose under the load in at least one round, by up to four milliseconds, and came back with real-time priority in every round. A tool times the wait for a core with everything else it times, so the wait reads as latency. Three tools are left out, because a wait for a core cannot be read in their figures: the Yahoo Streaming Benchmark's, set by its flush; PerfKit's, set by a managed queue's tail; and the Pub/Sub load test's, printed as buckets some 13 ms wide.

Kafka's own end-to-end tool, beside an observer. Kafka's end-to-end tool publishes each message, waits for its acknowledgment, and only then polls for the message, so it cannot time a message earlier than its acknowledgment. Version 35 (`freezes/32-experiment-plan`) ran a copy of the tool changed only to write down, for each message, its offset and the moments it began, was acknowledged and was polled. Beside it ran an observer on the same machine and clock, at real-time priority, that wrote down when each message became readable to it. The test, K1, ran six rounds of four cells, idle and with every core 88% busy, the tool at normal and at real-time priority, 3,000 messages each (Table S24). In every round some messages were readable before the tool had their acknowledgment: 0.5–4.4% idle, 11–41% under the load, and at most 0.2% with the tool at real-time priority. The three predictions version 35 froze after a pilot of two rounds held in all six: such messages occur under the load, the load raises their share, and real-time priority takes it back. The tool timed each such

TABLE S23
ON A BUSY MACHINE EVERY LISTED TOOL'S TAIL LENGTHENED, AND ITS MEDIAN BARELY MOVED IN ALL BUT FOUR. T3: EACH TOOL IDLE AND WITH EVERY CORE 88% BUSY; REAL-TIME: THE PROCESS THAT SUBTRACTS RAN AHEAD OF THE LOAD AT REAL-TIME PRIORITY. EACH FIGURE IS A CELL'S MEDIAN OVER THE FOUR ROUNDS, IN MILLISECONDS; THE TEXT'S CHANGES COMPARE THE CELLS WITHIN EACH ROUND.

	Median		99th percentile		
Tool	idle	88%	idle	88%	88%, real-time
Kafka end-to-end	1.00	1.00	2.00	7.00	2.00
Kafka producer	1.00	1.00	2.00	5.50	2.00
Kafka Trogdor	2.00	3.00	3.00	8.00	6.00
Artemis perf	1.16	0.88	1.59	5.17	3.89
BookKeeper benchmark	25.87	25.97	33.56	33.34	30.53
hey	0.80	0.80	2.00	3.20	2.00
k6	0.72	0.72	0.85	2.95	0.85
kgo-repeater	1.09	1.07	1.26	3.56	1.38
memtier	0.69	0.66	0.82	3.22	0.74
NATS bench	1.31	1.33	1.39	4.22	3.94
NATS latency	0.67	0.68	0.76	3.21	0.74
OpenMessaging Benchmark	2.00	2.00	2.00	6.00	5.00
Pravega benchmark	1.00	1.50	3.00	5.00	3.00
Pulsar perf.	3.00	3.00	4.00	8.00	6.00
Quiver	1.00	2.50	1.50	8.00	4.00
RabbitMQ PerfTest	0.85	0.84	1.02	3.81	1.02
Sarama producer	1.00	1.00	2.65	3.40	2.50
SBK	5.00	6.50	17.00	21.50	18.00
Stream PerfTest	1.00	1.00	2.00	5.00	4.00
Strimzi Canary	1.00	1.00	1.00	3.00	1.00
Valkey benchmark	0.51	0.53	0.54	2.41	0.56
Vegeta	0.76	0.76	1.60	3.52	1.56
wrk2	1.38	1.45	1.93	4.44	1.71
Xinfra Monitor	–	–	2.00	5.00	2.00
YCSB	0.73	0.63	0.86	2.93	0.77

TABLE S24
UNDER LOAD, 11–41% OF THE MESSAGES KAFKA'S END-TO-END TOOL TIMED WERE READABLE BEFORE IT HAD THEIR ACKNOWLEDGMENT. K1: THE TOOL'S TRACED COPY BESIDE AN OBSERVER AT REAL-TIME PRIORITY, SIX ROUNDS OF 3,000 MESSAGES PER CELL. READABLE FIRST: THE SHARE OF MESSAGES THE OBSERVER COULD READ BEFORE THE TOOL HAD THEIR ACKNOWLEDGMENT, WITH ITS RANGE OVER THE ROUNDS. S: THE OBSERVER'S ARRIVAL LESS THE TOOL'S ACKNOWLEDGMENT, EACH RUN'S MEDIAN. 99TH PERCENTILE: THE TOOL'S OWN, IN THE WHOLE MILLISECONDS IT PRINTS, AND THE OBSERVER'S, FROM EACH MESSAGE'S START TO ITS ARRIVAL. EACH FIGURE BUT THE RANGE IS THE MEDIAN OVER THE ROUNDS.

		Readable first (%)			99th percentile (ms)	
Load	Kafka's tool	median	range	S, median (ms)	tool	observer
idle	normal priority	0.9	0.5–4.4	0.19	3	1.99
idle	real-time priority	0.1	0.1–0.2	0.20	3	2.02
88%	normal priority	19.9	10.7–41.3	0.21	10	4.60
88%	real-time priority	0.1	0.0–0.2	0.22	3	2.11

message to a poll after its acknowledgment, a median 2.6 ms after the observer could read it, and under the load its own 99th percentile was 10 ms where the observer's was 4.6 ms. What the tool reports is the publish, the acknowledgment and the poll, one after another: a waiting thread lengthens it, and a message that overtook its acknowledgment cannot shorten it.

The step test. A tool whose two timestamps come from one clock meets a value below zero only if that clock steps back between them, as it did for a user of Stream PerfTest during a cluster's rolling restart. T2's constant offset cannot make one, so version 37 (`freezes/34-experiment-plan`) added S1. S1's tools ran once as they are and once while the driver's wall clock stepped back by 20 ms, or 50 ms where their trip is longer, at gaps drawn around a fifth of a second, with chrony stopped. A message on its way when the clock steps back is timed short by the step, and comes out below zero if the step is longer than its trip. Once the steps were undone, the wall clock was within 0.1 ms of the hypervisor's PTP clock. Every prediction versions 37 to 41 fixed from the tools' sources held in all four rounds except Sarama's and the Pub/Sub load test's. Each of the six tools on one clock that the steps reach, PerfKit, Strimzi Canary, Stream PerfTest, Xinfra Monitor, SBK and the Pravega benchmark, showed a value below zero its own way in every stepped run, and none in its controls (Table S25). The Pravega benchmark's statistics task threw at the first such value, losing every later statistic, and its code then exits with status 0, as after printing any exception, so a run that lost its statistics ends as one that succeeded. Kafka's end-to-end tool and `mqtt-benchmark`, on a monotonic clock, did not see the steps. Sarama's producer tool, also on a monotonic clock, kept its median, but its stepped 99th percentile fell 1.3 to 3.8 ms below its control's, so S1-i, which allowed a millisecond either way, failed in every round. The millisecond allowed was narrower than that figure's spread from one run to the next.

TABLE S25
WITH THE CLOCK STEPPED BACK UNDER THEM, SIX TOOLS THAT READ ONE CLOCK EACH HANDLED A VALUE BELOW ZERO THEIR OWN WAY. S1: EACH TOOL ONCE AS IT IS AND ONCE WHILE THE DRIVER'S WALL CLOCK STEPPED BACK BY THE STEP SHOWN AT GAPS DRAWN AROUND A FIFTH OF A SECOND, FOUR ROUNDS. STEPS: HOW MANY A STEPPED RUN MADE, LOWEST AND HIGHEST ROUND. BELOW ZERO: HOW MANY VALUES BELOW ZERO THE TOOL'S OWN OUTPUT SHOWED IN A STEPPED RUN, LOWEST AND HIGHEST ROUND, AND THE MOST IT SHOWED IN ANY CONTROL; THE PRAVEGA BENCHMARK'S ROW COUNTS THE ONE THAT STOPPED ITS STATISTICS, AND QUIVER'S ITS TRANSFERS, AS IT CALLS MESSAGES.

	Step		Below zero		
Tool	(ms)	Steps	stepped	controls	What its output shows
PerfKit messaging	50	363–397	8–18	0	samples below zero, in its mean and percentiles
Strimzi Canary	20	342–363	10–20	0	durations below zero, in its log
Stream PerfTest, one instance	20	302–315	16–25	0	latencies not kept, uncounted
Xinfra Monitor	20	506–517	40–84	0	values pinned to zero, in a debug line
Kafka end-to-end	20	29–33	—	—	median and 99th percentile within 1 ms of its control's
SBK	20	306–317	26–58	0	latencies it calls invalid, counted
mqtt-benchmark	20	297–308	—	—	minimum above zero, mean within 1 ms of its control's
Sarama producer	20	300–305	—	—	median as its control's, 99th percentile below it
Artemis perf	20	451–466	0	0	its producer stopped short, saying success
Quiver	20	326–343	9–25	0	values below zero in its transfers, its maximum wrapped
Pravega benchmark	20	335–341	1	0	no total printed, and an exception
Pub/Sub load test	50	1719–1737	—	—	nothing: binned with its fastest

Artemis's producer, which reads two clocks with its consumer, published 2 to 10 of its 3,000 messages in each stepped run, stopped, and reported success. A step inside one of its timed publishes makes a value its histogram refuses, in a thread that catches only the messaging library's own exceptions. Version 41 predicted that the Pub/Sub load test's stepped median would lie in its control's bucket or one either side (S1-l). That failed in one of the four rounds, round 2, where the stepped median lay two buckets below its control's even without the 17 values the steps put into the first bucket: Pub/Sub had delivered faster in that stepped run than in its control minutes before.

Every verdict and number here is recomputed from the per-run tables in `docs/results/tools/` by the judging programs, which the postmortem's S25.3 names with every file, the readings made after the runs and each block's write-up.

Public reports. Public reports rarely carry the deletion's signature, and no report we found says how many samples it kept. The audit behind that sentence was registered, committed and public before any report was read (`freezes/29-published-results-audit`). It fixed the signature, the places to search, the rule for inclusion, the coding and the outcomes, and nothing in it changed afterwards. The primary signature is a printed end-to-end median of exactly 1 ms, the smallest value the benchmark can print and the value every retained sample takes when the true end-to-end latency is shorter than the resolution. The strong signature adds a p99 of exactly 1 ms. Neither says a reported number is wrong, only that it came from an unstated fraction of the samples. The search covered Google Scholar, Semantic Scholar, the web, the engineering blogs of the vendors whose brokers the benchmark has drivers for, and the benchmark's repository and public copies, for reports up to 28 September 2026. A report was included if it named the benchmark or a fork as its tool and printed at least one numeric end-to-end percentile.

The audit included 43 reports, printing 1,100 configurations, of which 868 print a median. The primary signature occurs in eight configurations in two reports, both public repository text: five are Redis rates in a notebook behind a 2023 journal paper that shows its medians only in a chart, and three are maximum-rate runs in three result files of a 2021 fork. The strong signature occurs nowhere, and no included report states its retention or a discard count. The rules did not foresee several patterns. 9 of the included reports print medians that are not whole milliseconds, two of them below a millisecond, which the original benchmark's clock cannot produce. They come from copies whose maintainers changed how the span is timed, and for Kafka and Pulsar those copies still take the publish timestamp in whole milliseconds and keep the positivity filter. Five more reports only look finer: the benchmark's histogram prints a whole millisecond above about 262 ms as the top of a bucket wider than a microsecond, such as 507.001 ms, and their copies time the span as the original does. One 2024 fork prints p95, p99 and p999 all at exactly 1 ms beside an average just above it and no median column, so by the registered rule it does not count. Four configurations print a median of zero, from empty histograms. The coding, the search log and every excluded report with its reason are in `docs/results/external/published_results_audit` (postmortem, S25.1).

One more pattern was read after the audit and is not one of its outcomes. The benchmark's RabbitMQ driver takes the publish timestamp from the message's AMQP timestamp property, which carries whole seconds, so every RabbitMQ end-to-end value it prints carries an extra zero to one second, set by where in its second the message was published. Two included reports print RabbitMQ percentiles spread evenly over a second, with medians near 500 ms and 99th percentiles near 990 ms, one at every rate and the other at light loads. This is the whole-second timestamp's signature, which the registered one, a median of 1 ms, could not see. A vendor's copy fixed the timestamp in 2020 and did not return the fix to the original.

S6. THE REGISTERED LAW CAMPAIGN

A law for the negative-span rate, fixed before its runs, had one prediction confirmed on every campaign that tested it, three on one broker only, and the rest not confirmed (Table S26). The one confirmed everywhere is that load raises the rate at the

shortest end-to-end latencies. Neither prediction of where the rate falls with the end-to-end latency, at a slice set by hand, was confirmed. The predictions of how quickly it falls held on one broker at most, and Python's rate exceeded the registered limit relative to Java's. Frozen programs judged each prediction over 8,102 runs on machines no earlier campaign used.

S6.1. The law, and the design that tested it

The registered law says that the scheduler's settings alone fix where the negative-span rate falls as the end-to-end latency lengthens, and how wide the fall is. It assumes that a thread kept waiting waits out a whole slice s of another task, and then up to one tick h. The rate then sits on a plateau $c + p$ while the end-to-end latency D is shorter than the slice, falls across one tick, and rests on a floor c beyond it:

$$r(D) = c + p \cdot \begin{cases} 1, & D \le s, \\ (s + h - D)/h, & s < D < s + h, \\ 0, & D \ge s + h. \end{cases} \tag{S2}$$

Here p is the waiting probability of Equation 3, and c is the floor that ordinary short delays leave. The slice and the tick predict where the fall starts and how wide it is; the plateau's height and the floor are measured. In the plan, a reading is negative when the thread that reads the clock for the acknowledgment waits longer than the margin $D - g$, a run's median end-to-end latency less its median publish latency g, the plan's letter for A. The campaign lengthens only the path to the consumer, so within a setup g stays put and the margin moves one for one with D.

The three Azure pairs of Table S2 ran the campaign. The plan's blocks A1 to A9 are written L1 to L9 here. L1 sets the slice on the first x86 pair, and L4 on the Arm pair. L2 sets the tick, on three kernels built from one source tree that differ in the tick alone: 1, 4 and 10 ms. L3 sets the load, L5 the number of online CPUs, L7 real-time priority for the threads that read the clock, and L8 the client, Kafka's own Java client against our Python one. L6 records every `python3` thread's waits in half of L1's and L3's runs, and L9 records each of those threads' scheduling events in half its own runs. Each block carries numbered predictions, stated and judged in Table S26. A further block, M0, on bursty traffic, bears on no claim here.

Each run replays messages at a steady 50 a second and measures its end-to-end latency, publish latency and negative-span rate after a 30 s warm-up. The end-to-end latency is set by delaying only the traffic to the receiver's own network address, so the acknowledgment's path is untouched. Each session first calibrates how far each added delay moves the end-to-end latency, and an end-to-end latency shorter than the client's own with nothing added cannot be set, and was not run. The integrity rule judges every run as it ends. A run counts when every condition it was meant to have took effect, from the load within 3 points of its target to the added delay within $50\,\mu$s of its setting. Otherwise it is repeated, up to three attempts. A stopping rule on the publish latency stopped a campaign whose median publish latency moved too far, until the final version made it only record. The plan's own name for the publish latency, and for the stopping rule on it, is "got-it".

A round runs every setup once. Each campaign that tests a prediction took its rounds, at least four and at most forty, from simulating its own decision rule on data that follow the law. L2 and L9, which have no rule the simulation can take, ran at the floor of four, and no campaign ran more than 24. A prediction about where the fall sits is read twice, on a free reading, a curve required only never to rise, and on the law's fitted shape, and it passes only where both agree. The judging read 8,102 runs in 156 campaigns, taken from 16 to 27 September 2026, and 7,996 of them counted.

The plan was first frozen as version 5 on 16 September 2026, before its first run, and the version in force at the end is the thirty-second. A frozen version is never edited. A change is a new, numbered freeze, public before the runs it covers, whose first section says what changed, why, and whether a result already seen prompted it. Three kinds of change followed the campaigns' own results. Version 14, after L3's first campaign could not place the fall at 50% load, judged P3a only where the plateau reaches 2%. The stopping rule was corrected five times, each time after it had stopped a campaign. And three versions corrected judging programs that had already judged, each stating what the verdicts it touches had been. Blocks added as the campaign ran, in versions 24 and 27 to 29, were each read by a program fixed before their runs, and P6 was given its program in version 30, at a margin no run had been read against. Every freeze is in the repository's `freezes/` folder, whose index says what each one changed and whether a result prompted it. The index also says how a reader can check that each freeze came before its runs, and what each check rests on: fingerprints that depend on the files alone, the commit each run records, and the time of each push and of the checks it started, which GitHub keeps for about ninety days. The history was last rewritten on 5 October 2026, which gave every earlier commit, every freeze's among them, a new identifier, and no archive dated by a third party yet holds a freeze.

S6.2. Load raised the rate everywhere, and the rate did not fall where the law put it

Load raised the rate at the shortest end-to-end latencies in all five load campaigns, all on Kafka, with every rise's interval above zero. Table S26 gives each verdict as its frozen program returned it. A prediction judged broker by broker is confirmed as a whole only if it holds on both brokers. On Kafka, real-time priority took the plateau to zero in every run, and at 8 CPUs, the one core count whose slice is at least Kafka's shortest end-to-end latency, the fall sat inside the band that slice predicts,

from the slice to a tick past it. Kafka's own Java client shows the same fall and the same effect of priority, at a plateau a third to a quarter of the Python client's. The Arm pair has values of S below zero as the x86 pairs do: P8 and P9 read a plateau and a fall there (Table S26), and every run of the Arm pair, with its rate, is drawn in the postmortem, S36.

On the first x86 pair the free reading placed the fall's halfway point inside its band at every slice each broker reaches, four for Kafka and five for Redis. The fitted shape did so at two of four slices for Kafka and two of five for Redis. On the Arm pair the free reading held on both brokers and the fitted shape failed on both (Figure S5). P1 and P9 need both readings, so neither is confirmed. Where several campaigns ran a block, each campaign's halfway points are moved by its own offset, so that its reading at the 3 ms slice, the anchor, comes out at the block's mean. P9 tests the band only at the slices a pair reaches, but takes its slope through every slice that has a halfway point, so Kafka's judged slopes on the Arm pair run through three slices, the shortest of them out of reach. Read after the verdicts, through the two slices in reach alone the free slope would lie inside the range P9 allows and the fitted slope just below zero. Load moved the fall toward shorter end-to-end latencies in every campaign, and P3a held in none.

TABLE S26
ONE PREDICTION CONFIRMED ON EVERY CAMPAIGN, THREE ON ONE BROKER, AND THE REST NOT. EVERY PREDICTION REGISTERED FOR BLOCKS L1 TO L9, THE NUMBER IT WAS JUDGED ON OVER ALL RUNS, AND ITS VERDICT, BY BROKER WHERE THE TWO DIFFER. INTERVALS IN PARENTHESES ARE 95% INTERVALS FROM RESAMPLING WHOLE ROUNDS.

	Block, pair	The prediction	On all runs	Verdict
P1	L1, first x86	the fall's halfway point inside $[s, s+h]$ at all but one of the slices a pair reaches, at least four of them; its slope on s in 0.8–1.2	slope, free and fitted: Kafka 0.758 (0.596–1.023) and 0.230; Redis 1.007 (0.844–1.169) and 0.662	not confirmed
P9	L4, Arm	on the Arm pair, inside at every slice it reaches, at least two; the same slope	Kafka 0.830 (0.627–1.172) and 0.285; Redis 1.153 (0.753–1.325) and 0.798	not confirmed
P2	L2	the fall 2.5 to 5.5 times wider at a 4 ms tick than at 1 ms	ratio, free and fitted: Kafka 1.784 and 1.818; Redis 1.064 and 1.126 (S6.3)	not confirmed
P2b	L2	6 to 14 times wider at a 10 ms tick	Kafka 4.786 and 5.358; Redis 3.045 and 3.339 (S6.3)	not confirmed
P2c	L2	the width on the tick a line through zero, slope 0.8–1.2	fitted slope: Kafka 0.968 (0.271–1.125); Redis 0.874 (0.807–1.034)	confirmed on Redis only
P2d	L2	the fitted start within 0.25 ms of the slice at every tick	within it at one of twelve slice and tick pairs	not confirmed
P3a	L3, five Kafka campaigns	the halfway point moves less than 0.25 ms from 75% to 88% load	free move: −0.009, −0.188, −0.086, −0.011 and −0.309 ms	not confirmed on any
P3b	L3, five Kafka campaigns	load raises the plateau	rise, free: 0.048 (0.044–0.052), 0.047, 0.048, 0.041 and 0.050	confirmed on all five
P4	L7, first x86	real-time priority cuts the plateau at least five times wherever it is at least 2%	Kafka 0.0405 to zero; Redis's plateau 0.0069 (S6.3)	confirmed on Kafka only
P6	L6	a recording of every thread's waits predicts the rate within 0.67–1.5	median ratio 2.49 to 13.08 over four pair and broker parts; no setup inside	not confirmed
P7	L5, second x86	the default slice follows the core count, and the fall follows it	slices read back at 2, 4 and 8 CPUs as the rule gives; the fall on Kafka at 8 CPUs, Redis's fitted shape missing at all three	confirmed on Kafka only
P8	L8, first x86 and Arm	Java has the same fall and priority effect, and Python's plateau is at most 1.5 times Java's	Python over Java: 3.251 (2.795–3.907), and 4.253 (4.013–4.560) on Arm	not confirmed
L9-1	L9, three pairs	at least 90% of negative readings mostly spent waiting for a CPU	79.4 to 92.0%	not confirmed
L9-2	L9	the time-weighted waits predict the rate within 0.67–1.5	median ratio 0.104 to 0.165	not confirmed
L9-2b	L9	the share of acknowledgments whose own wait outlasted $D - g$ predicts it	inside the rule on four of six parts; median ratio 1.11 to 1.30	not confirmed

The plan keeps every run, and re-runs none whose conditions held. Void runs, runs the integrity rule did not count, and the 18 runs of six false starts, each false start replaced at once by a campaign of the same design, are judged by no program. Four marks leave a run counted: flagged, paused, far from its setup's other repeats, and traced. The stopping rule would have stopped the 14 flagged runs, taken while it only recorded. In the 222 paused runs a message arrived more than 150 ms late, either because the receiving side's disk stalled, for a cause not identified, or because Kafka's broker could not reach its own controller. No run had a message arrive before it was published. Without the flagged runs, or without all 1,110 taken while the stopping rule only recorded, no verdict moves. Without the paused runs every verdict stands but P3a on the first x86 pair, whose fitted interval sits on the edge of its ± 0.25 ms bar, just outside it with them and just inside without. The verdict on all the runs is the one reported. The verdicts, on all runs and on each subset, are under `docs/results/law/judged-27-sep`.

S6.3. Read after the verdicts: the wait is the rest of a slice, and two judging programs departed from the plan

None of what follows, read after the verdicts, changes one. The rate holds no plateau: at half the slice it is 1.45 to 3.94 times its value at 0.9 of the slice, in all ten slice and broker combinations whose runs reach half the slice.

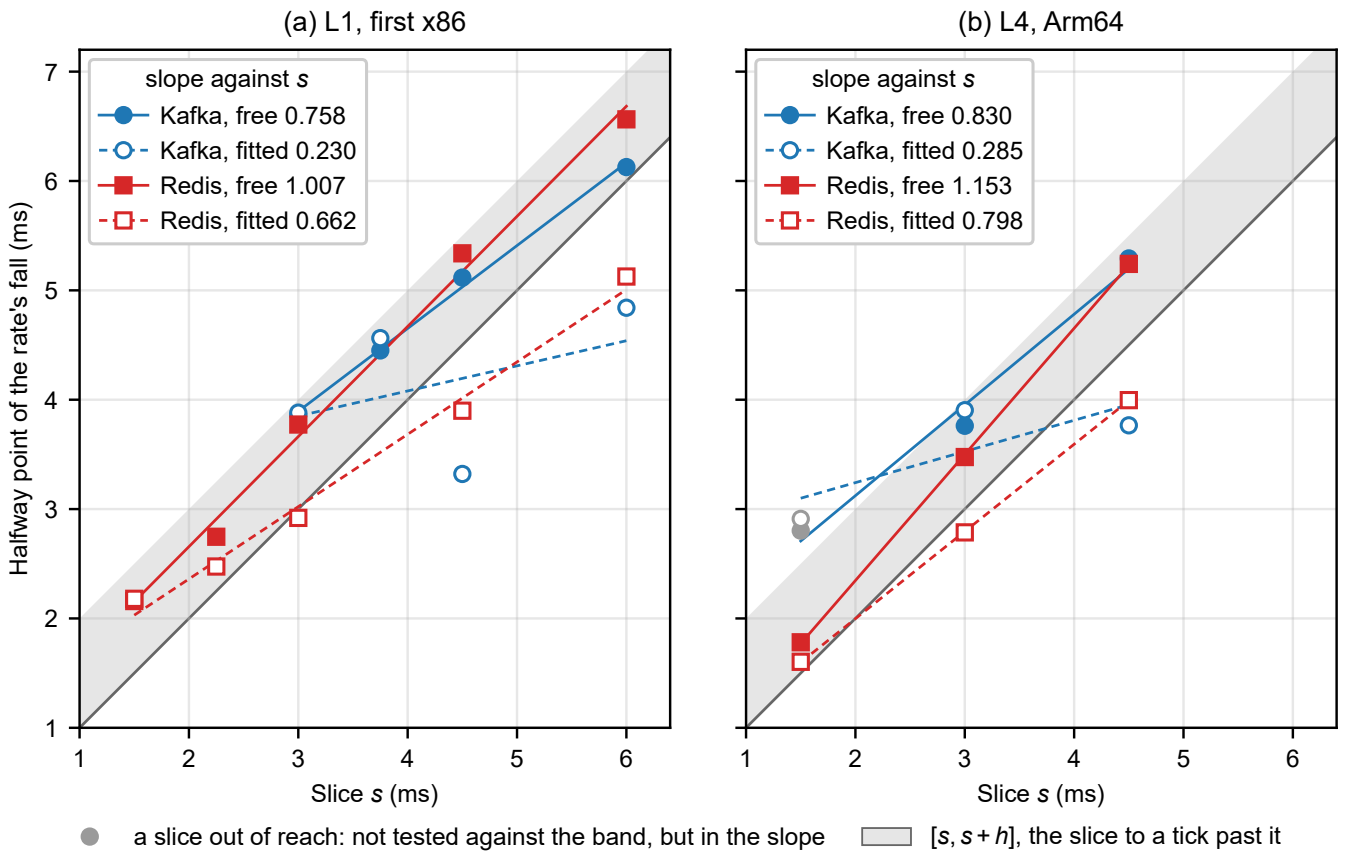


Fig. S5. **Each slice's fall lies inside its band on the free reading, and on the fitted shape at fewer than half of the slices.** For L1 (first x86 pair) and L4 (Arm pair), each broker's halfway point of the rate's fall at each slice, read as the frozen judges read it, with each campaign moved by its own offset through the 3 ms anchor. Filled: the free reading. Open: the fitted shape. Gray band: $[s, s+h]$, from the slice to a tick past it ($h = 1$ ms); its lower edge is the 1:1 line. Each line is the straight line through a broker's points whose slope P1 and P9 judge, and the legends give the slopes (Table S26). Gray points: Kafka's 1.5 ms slice on the Arm pair, out of reach for the band test but in the judged slope. Every run of L1 and L4, aberrant runs marked, is in the postmortem's Part V.

L9's recordings, at a 3 ms slice and a 1 ms tick, time the wait of the thread that reads the clock for each acknowledgment, for 1,516,342 acknowledgments in 304 runs. Of those whose thread waited more than 1 ms, 24.3 to 35.8% were still waiting at 3 ms and 3.3 to 6.8% past 4 ms, over twelve pair, broker and load parts. A thread preempted for a task starting its slice waits out all of it, but a thread woken while another task holds the core waits out what is left, and turning off the kernel's run-to-parity rule did not shorten that wait (S3.11). The acknowledgment's reply wakes the thread that reads it, so its wait is the second kind. A whole-slice wait would leave all of them still waiting at 3 ms; a wait for the rest of the slice leaves 20% waiting at 3 ms and none past 4 ms. That thread waits out what is left of the running task's slice, R, uniform on $[0, s]$, and then up to a tick, U, uniform on $[0, h]$, so that for $h \le s$

$$G(x) = \Pr[R + U > x] = \begin{cases} 1 - x^2/(2sh), & 0 \le x \le h, \\ 1 - (x - h/2)/s, & h \le x \le s, \\ (s + h - x)^2/(2sh), & s \le x \le s + h, \\ 0, & x \ge s + h. \end{cases} \tag{S3}$$

The recording raised the rate it recorded, 1.36 to 1.49 times for Kafka and 3.69 to 4.33 times for Redis, so these waits describe the machines with it on. Fitted to a rate that falls from the shortest end-to-end latency on, the law's shape puts its fall below the slice and moves it less than the slice moves, which is why the free and fitted readings of P1 and P9 split.

In two places a frozen program departed from the plan, and in both the frozen answer is the one reported. The programs judging P2, P2b and P2c group L2's runs by tick alone, so each width they read spans the falls of both its slices, 1.5 and 3 ms. With Equation S2 in place of every run's rate, as if the law held exactly, the P2 program returns 1.98 free and 1.57 fitted for Redis, below its band. One slice at a time, the same formula gives 4.0 to 5.2 for P2 and 9.8 to 12.0 for P2b, inside both bands. Those three verdicts therefore carry no evidence about the law either way. P4's program words Redis "not confirmed" although its plateau, 0.69%, lies below the 2% under which the plan's rule reports a plateau as out of reach.

REFERENCES

[S1] StatsBomb, "Statsbomb open data," *GitHub Repository*, 2023, accessed: Oct. 15, 2026. [Online]. Available: https://github.com/statsbomb/open-data

[S2] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[S3] D. Katz, J. Baptista, S. P. Azen, and M. C. Pike, "Obtaining confidence intervals for the risk ratio in cohort studies," *Biometrics*, vol. 34, no. 3, pp. 469–474, 1978.

[S4] T. Zuo, J. Hwang, A. Tang, R. Agarwal, and Q. Cai, "Understanding host network stack latency," in *Proc. ACM SIGCOMM 2026 Conf.*, Aug. 2026, pp. 1127–1140.

[S5] Linux Kernel Documentation, "KVM-specific MSRs: `MSR_KVM_SYSTEM_TIME_NEW`," https://docs.kernel.org/virt/kvm/x86/msr.html, 2026, accessed: Aug. 21, 2026.

[S6] ——, "Clocks and timers (Hyper-V)," https://docs.kernel.org/virt/hyperv/clocks.html, 2026, accessed: Oct. 15, 2026.

[S7] S. MacMullen, "[rabbitmq-discuss] messages are read by consumer befor than sender get confirmation," rabbitmq-discuss mailing list, https://lists.rabbitmq.com/pipermail/rabbitmq-discuss/2014-June/036711.html, Jun. 2014, accessed: Oct. 15, 2026.

[S8] Z. Zhou, "sched: Reduce the default slice to avoid tasks getting an extra tick," Linux kernel source, commit 2ae891b82695, first released in v6.15, 2025, https://github.com/torvalds/linux/commit/2ae891b826958b60919ea21c727f77bcd6ffcc2c. Accessed: Oct. 15, 2026.

[S9] Linux Kernel Documentation, "Timestamping," https://docs.kernel.org/networking/timestamping.html, 2026, accessed: Oct. 15, 2026.

[S10] ——, "Reducing OS jitter due to per-cpu kthreads: `ksoftirqd`," https://docs.kernel.org/admin-guide/kernel-per-CPU-kthreads.html, 2026, accessed: Oct. 15, 2026.

[S11] N. Zilberman, M. Grosvenor, D. A. Popescu, N. Manihatty-Bojan, G. Antichi, M. Wójcik, and A. W. Moore, "Where has my time gone?" in *Passive and Active Measurement (PAM 2017)*, ser. Lecture Notes in Computer Science. Springer, 2017, pp. 201–214.

[S12] Hewlett-Packard, "Time interval averaging," Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766.

[S13] Apache Software Foundation, "Apache ActiveMQ artemis — `artemis perf`," Source repository, 2026, https://github.com/apache/artemis. Accessed: Oct. 11, 2026.

[S14] ——, "Apache BookKeeper — `BenchThroughputLatency.java`," Source repository, 2026, https://github.com/apache/bookkeeper. Accessed: Oct. 11, 2026.

[S15] ——, "Apache Kafka tools — `EndToEndLatency.java` and `ProducerPerformance.java`," Source repository, 2026, https://github.com/apache/kafka. Accessed: Aug. 31, 2026.

[S16] Apache Pulsar contributors, "`PerformanceConsumer.java`," Source repository, 2026, https://github.com/apache/pulsar. Accessed: Aug. 21, 2026.

[S17] Apache Software Foundation, "Apache RocketMQ — `example/.../benchmark/Consumer.java`," Source repository, 2026, https://github.com/apache/rocketmq. Accessed: Oct. 11, 2026.

[S18] Google, "Cloud pub/sub load test framework — `MetricsHandler.java`," Source repository, 2026, https://github.com/GoogleCloudPlatform/pubsub. Accessed: Oct. 11, 2026.

[S19] IBM, "Sarama — `broker.go` and `tools/kafka-producer-performance`," Source repository, 2026, https://github.com/IBM/sarama. Accessed: Oct. 11, 2026.

[S20] Apache Software Foundation, "Apache Kafka trogdor — `TimestampRecordProcessor.java`," Source repository, 2026, https://github.com/apache/kafka. Accessed: Oct. 11, 2026.

[S21] Synadia, "NATS cli — `cli/latency_command.go`," Source repository, 2026, https://github.com/nats-io/natscli. Accessed: Oct. 2, 2026.

[S22] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/. Accessed: Aug. 19, 2026.

[S23] Google, "Perfkit benchmarker — `messaging_service_scripts`," Source repository, 2026, https://github.com/GoogleCloudPlatform/PerfKitBenchmarker. Accessed: Oct. 11, 2026.

[S24] Pravega, "Pravega benchmark — `PerfStats.java`," Source repository, 2026, https://github.com/pravega/pravega-benchmark. Accessed: Oct. 11, 2026.

[S25] J. Ross, "Quiver — `python/quiver/arrow.py`," Source repository, 2026, https://github.com/ssorj/quiver. Accessed: Oct. 11, 2026.

[S26] Broadcom, "RabbitMQ PerfTest — `TimestampProvider.java` and `DefaultPerformanceMetrics.java`," Source repository, 2026, https://github.com/rabbitmq/rabbitmq-perf-test. Accessed: Oct. 2, 2026.

[S27] ——, "Rabbitmq stream perftest — `DefaultPerformanceMetrics.java`," Source repository, 2026, https://github.com/rabbitmq/rabbitmq-stream-perf-test. Accessed: Oct. 11, 2026.

[S28] Redpanda Data, "kgo-verifier — `pkg/worker/repeater/repeater_worker.go`," Source repository, 2026, https://github.com/redpanda-data/kgo-verifier. Accessed: Oct. 11, 2026.

[S29] IOP Systems, "Rezolus: systems performance telemetry — `scheduler_discarded_samples`," Source repository, 2026, https://github.com/iopsystems/rezolus. Accessed: Aug. 20, 2026.

[S30] K. Munegowda, "SBK: Storage benchmark kit — `LatencyRecorder.java`," Source repository, 2026, https://github.com/kmgowda/SBK. Accessed: Oct. 11, 2026.

[S31] Strimzi, "Strimzi canary — `internal/services/consumer.go`," Source repository, 2026, https://github.com/strimzi/strimzi-canary. Accessed: Oct. 11, 2026.

[S32] LinkedIn, "Xinfra monitor — `ConsumeService.java`," Source repository, 2026, https://github.com/linkedin/kafka-monitor. Accessed: Oct. 11, 2026.

[S33] Yahoo, "Yahoo streaming benchmark — `data/src/setup/core.clj`," Source repository, 2026, https://github.com/yahoo/streaming-benchmarks. Accessed: Oct. 11, 2026.

[S34] A. D. Brunelle, "`blktrace`/`btt` — `btt/inlines.h`," Source repository, 2026, accessed: Aug. 20, 2026.

[S35] EMQ Technologies, "`emqtt-bench`: MQTT benchmark tool — `src/emqtt_bench.erl`," Source repository, 2026, https://github.com/emqx/emqtt-bench. Accessed: Aug. 21, 2026.

[S36] J. Axboe, "fio: Flexible I/O tester — `gettime.c`," Source repository, 2026, https://github.com/axboe/fio. Accessed: Aug. 20, 2026.

[S37] Confluent, "`librdkafka` — `examples/rdkafka_performance.c`," Source repository, 2026, https://github.com/confluentinc/librdkafka. Accessed: Oct. 11, 2026.

[S38] A. Krylovsky, "mqtt-benchmark — `client.go`," Source repository, 2026, https://github.com/krylovsk/mqtt-benchmark. Accessed: Oct. 11, 2026.

[S39] G. Tene, "`wrk2` — `src/wrk.c`," Source repository, 2026, https://github.com/giltene/wrk2. Accessed: Aug. 21, 2026.